SYDE 671 · ASSIGNMENT 2 · FALL 2026

Spatial Filtering and Multiresolution Blending

Moksh Trehan · October 19, 2026

Experiments in convolution, image gradients, sharpening, hybrid images, and multiresolution blending. Each section presents the method, visual results, and observations.

Contents

1. **Part One · Filters**
    1. Convolutions from scratch
    2. Finite differences and thresholding
    3. Derivatives of Gaussians
    4. Gradient orientations
2. **Part Two · Frequencies**
    1. Image sharpening
    2. Hybrid images
    3. Gaussian and Laplacian stacks
    4. Multiresolution blending

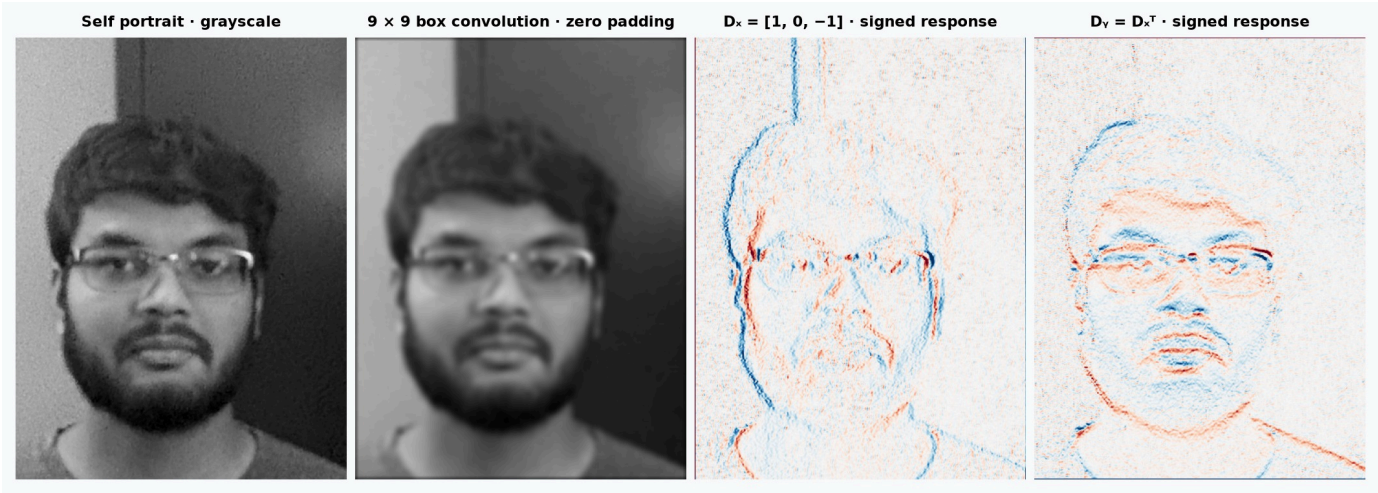
Implementation appendix · Reproduction · Image credits

Online report: https://mokshtrehan.github.io/syde671-assignment2/

1.1 Convolutions from scratch

Convolution flips a kernel in both dimensions, then sums the products of overlapping samples. I implemented this operation using four loops and two loops, and applied a box filter and two derivative filters to the self portrait.

Filters $B = (1/81) \cdot \mathbf{1}_{9\times9}$ $D_x = [1, 0, -1]$ $D_y = D_x^T$



Self portrait, box-filtered image, and signed horizontal and vertical derivatives. Red indicates positive responses and blue indicates negative responses. The shared derivative scale clips the strongest responses for display.

Filtering the portrait

The box filter removes fine texture. The horizontal and vertical derivatives emphasize intensity changes across columns and rows. Their signed responses share a symmetric color scale. Zero padding darkens the outer rim of the smoothed image and can introduce derivative responses at the frame.

The working image is 384 × 512 pixels. Grayscale intensity is 0.299R + 0.587G + 0.114B, with RGB values in [0, 1].

Implementation

The four-loop version iterates over image rows, image columns, kernel rows, and kernel columns. The two-loop version retains the kernel loops and multiplies complete shifted image slices. Both use NumPy operations only; neither calls a convolution, FFT, window-view, or image-processing routine.

Appendices A and B contain the input checks, padding, and both complete implementations.

1.1 Runtime and boundary effects

Both implementations agree with SciPy to floating-point precision. Non-symmetric odd and even kernels test kernel reversal and output alignment; a symmetric Gaussian alone would not reveal either error.

Runtime and numerical agreement · 96 × 96 float64 input

Kernel	Method	Median time (ms)	Max. absolute error
3×5 asymmetric	Four loops	43.908	5.55e-16
3×5 asymmetric	Two loops	0.123	5.55e-16
3×5 asymmetric	SciPy convolve2d	0.209	0.00e+00
2×4 asymmetric (even)	Four loops	23.082	8.88e-16
2×4 asymmetric (even)	Two loops	0.08	8.88e-16
2×4 asymmetric (even)	SciPy convolve2d	0.138	0.00e+00
9×9 box	Four loops	209.111	7.77e-16
9×9 box	Two loops	0.609	7.77e-16
9×9 box	SciPy convolve2d	0.9	0.00e+00
1×3 derivative	Four loops	9.022	0.00e+00
1×3 derivative	Two loops	0.047	0.00e+00
1×3 derivative	SciPy convolve2d	0.095	0.00e+00

Both manual methods require O(HWKhKw) arithmetic. The two-loop version removes Python's per-pixel loop overhead by operating on image slices. These timings are local measurements, using a 96 × 96 float64 image and the median of three calls after one warm-up.

What happens at the image boundary?

Boundary values for a constant image

SciPy boundary setting	Corner	Top edge midpoint	Center
fill	0.308642	0.555556	1.0
symm	1.0	1.0	1.0
wrap	1.0	1.0	1.0

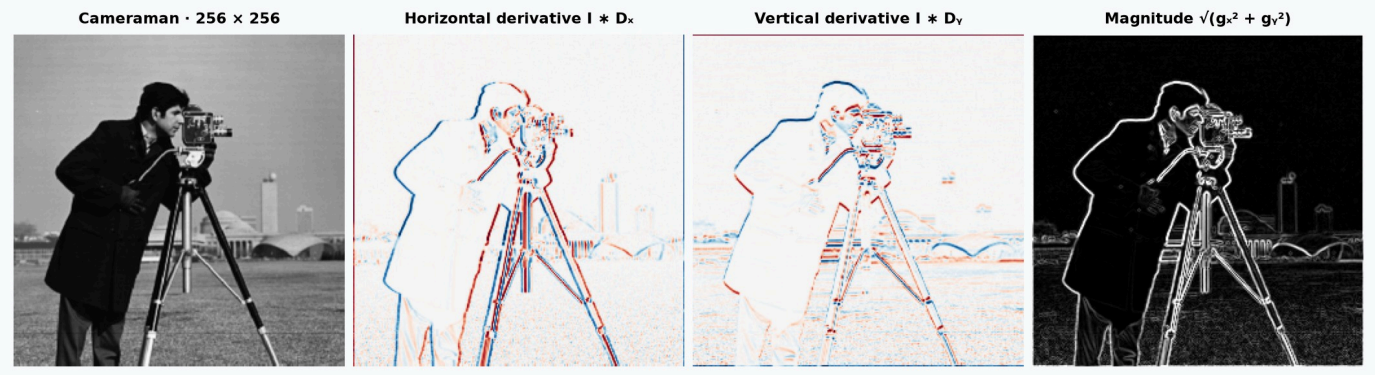
For a constant 25 × 25 image and a normalized 9 × 9 box filter, zero padding gives 25/81 at a corner and 5/9 at an edge midpoint. Symmetric and periodic extension preserve one. The "same" option sets output size; the boundary option supplies samples outside the image.

Along each axis, the manual implementations pad by floor(K/2) before the image and floor((K–1)/2) after it. This matches SciPy's "same" convention for even kernels. The largest comparison error is 8.88×10^{-16}.

1.2 Finite difference operator

The cameraman image is differentiated horizontally and vertically. Gradient magnitude combines these signed responses into an unsigned measure of edge strength.

Gradient $g_x = I * D_x \quad g_y = I * D_y \quad |\nabla I| = \sqrt{(g_x^2 + g_y^2)}$



Cameraman finite differences and gradient magnitude. Signed derivatives share the range [−0.65, 0.65]; magnitude uses [0, 0.65].

Interpreting the derivatives

The horizontal response emphasizes intensity changes across columns; the vertical response emphasizes changes across rows. Convolution reverses the derivative kernel, so an increase from left to right produces a positive horizontal response.

The white border is cropped from the supplied cameraman image before resizing it to 256 × 256 pixels. Computation uses floating-point intensities in [0, 1]. The complete crop coordinates and preprocessing steps are recorded in Appendix B.

1.2 Choosing an edge threshold

A pixel is classified as an edge when its gradient magnitude exceeds T. Three thresholds show the balance between retaining weak contours and rejecting background texture.



Threshold comparison for the finite-difference magnitude. The selected threshold is 0.16. The black outer frame is excluded from evaluation.

Threshold sensitivity

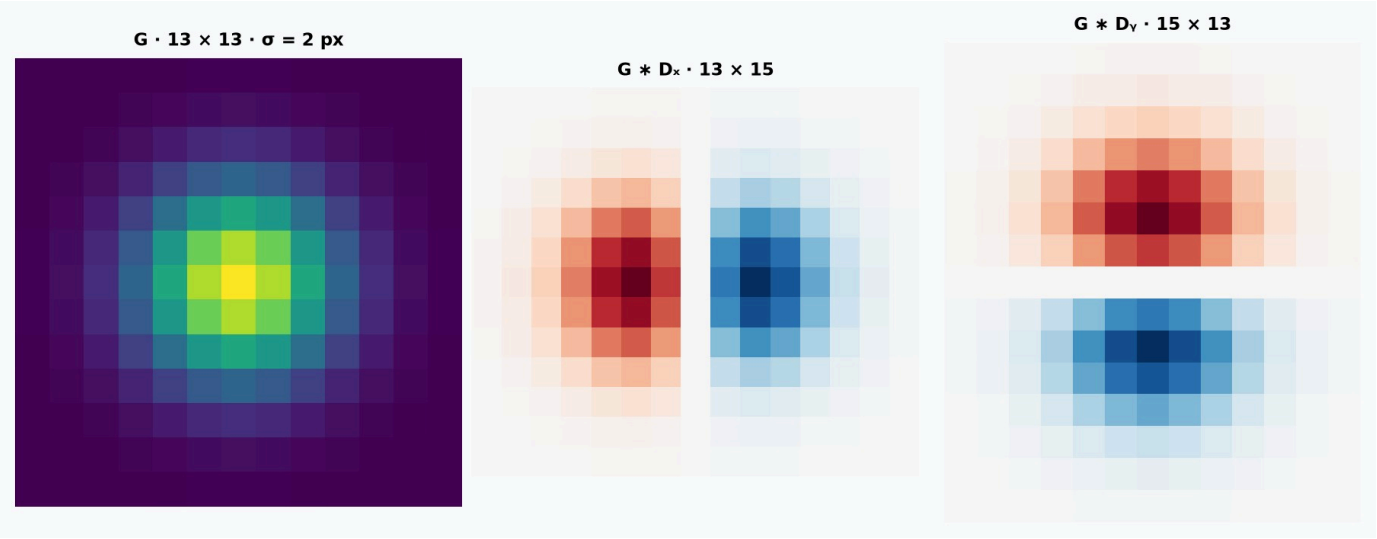
Threshold	Interior edge fraction	Selection
0.08	23.68%	Comparison
0.16	12.81%	Selected
0.24	8.99%	Comparison

A threshold of 0.16 preserves the person, camera, and tripod while rejecting much of the ground texture visible at 0.08. Raising it to 0.24 removes more weak contours. This selection is based on the images; no annotated edge map is used to measure detection accuracy.

A seven-pixel frame is excluded from all Part 1 binary maps and edge-fraction measurements. This removes responses caused by zero padding and covers the largest Gaussian-derivative support. The signed derivative and magnitude images show the complete responses.

1.3 Derivative of Gaussian

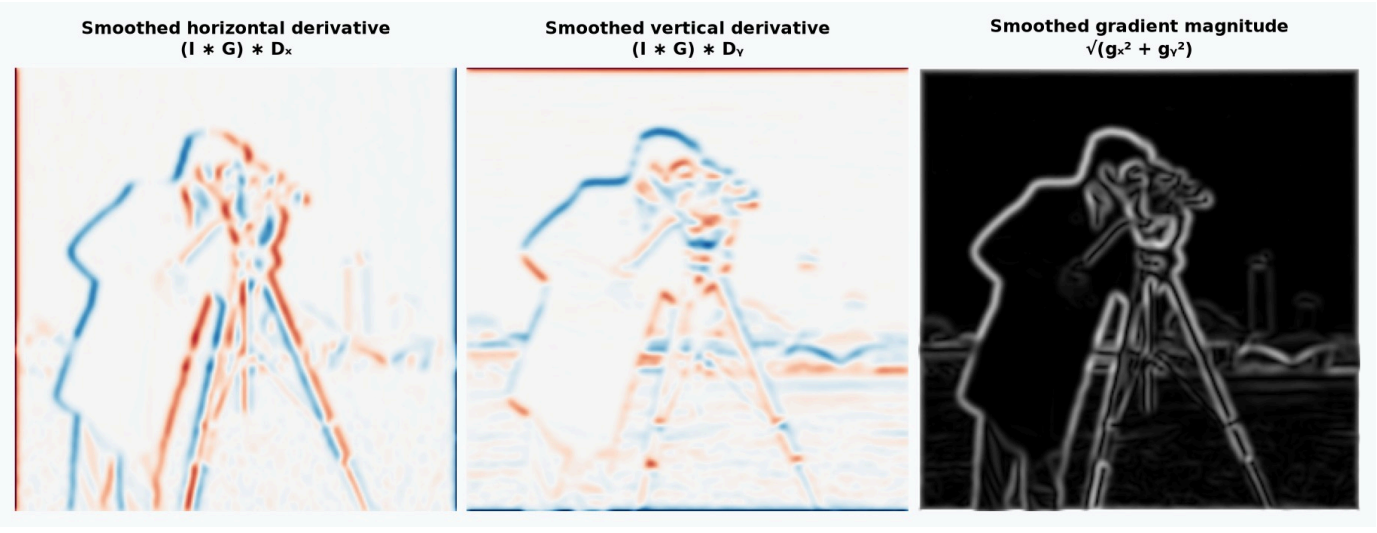
Smoothing before differentiation reduces the response to fine texture. A 13 × 13 Gaussian with σ = 2 pixels is formed from the outer product of cv2.getGaussianKernel(13, 2) with itself.



Gaussian kernel and its sampled horizontal and vertical derivatives. The derivative kernels share a signed scale; the Gaussian uses a separate nonnegative scale.

Convolving this Gaussian with the finite-difference kernels gives a single filter for each derivative. Full kernel convolution produces supports of 13 × 15 and 15 × 13 pixels.

Combined filters $K_x = G * D_x$ $K_y = G * D_y$



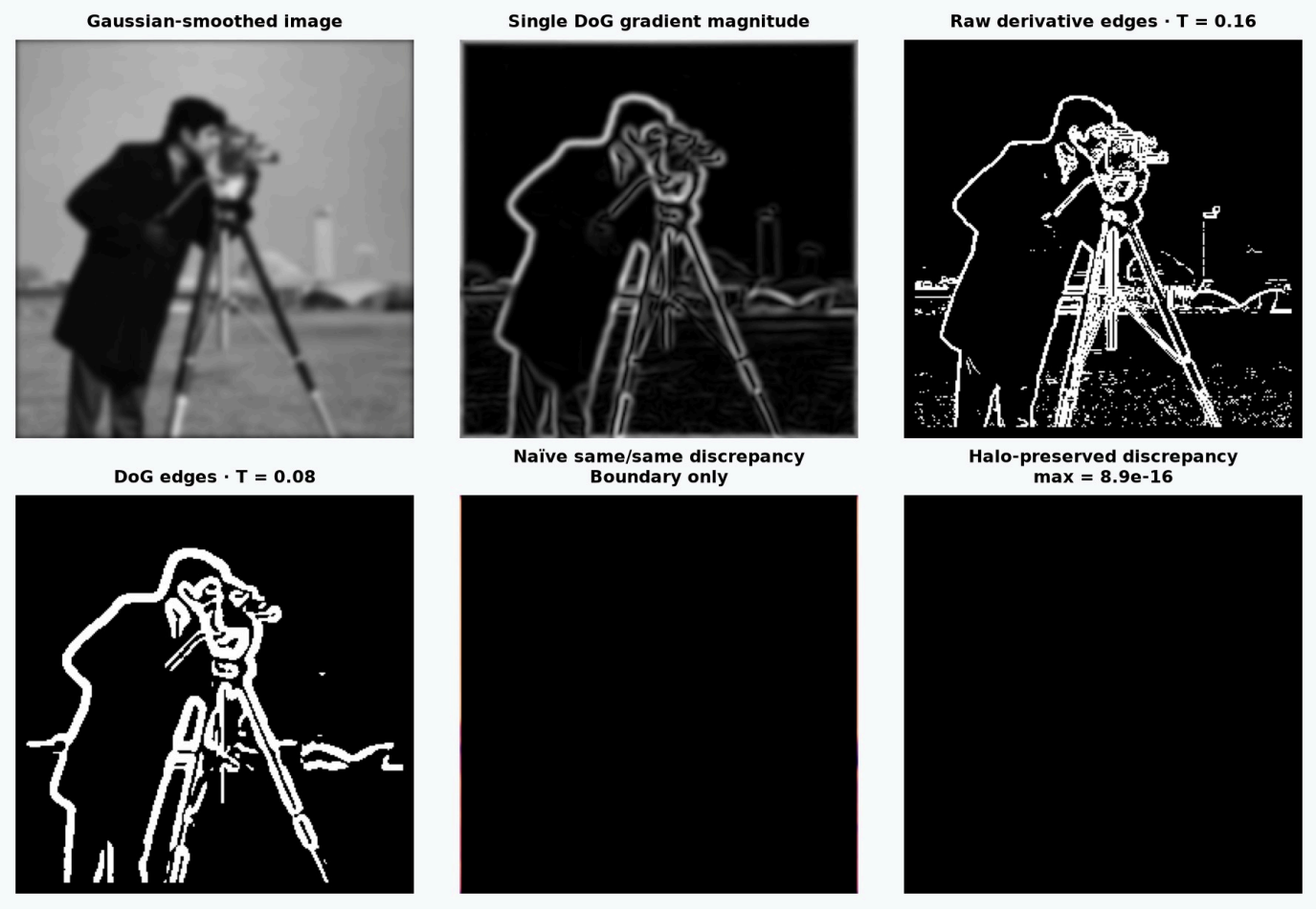
Finite differences after Gaussian smoothing, using two successive “same” convolutions with zero padding. Signed derivatives share [−0.35, 0.35]; magnitude uses [0, 0.35].

Compared with the unsmoothed derivatives, the contours are wider and the grass texture is weaker. Nearby edges can merge, and weak detail can disappear.

1.3 Sequential and combined filtering

Sequential and combined filtering should agree when both use the same boundary treatment.

Associativity $I * (G * D) = (I * G) * D$



Gaussian derivative results, thresholded edges, and boundary differences. Keeping the full intermediate blur restores agreement.

Agreement between the two formulations

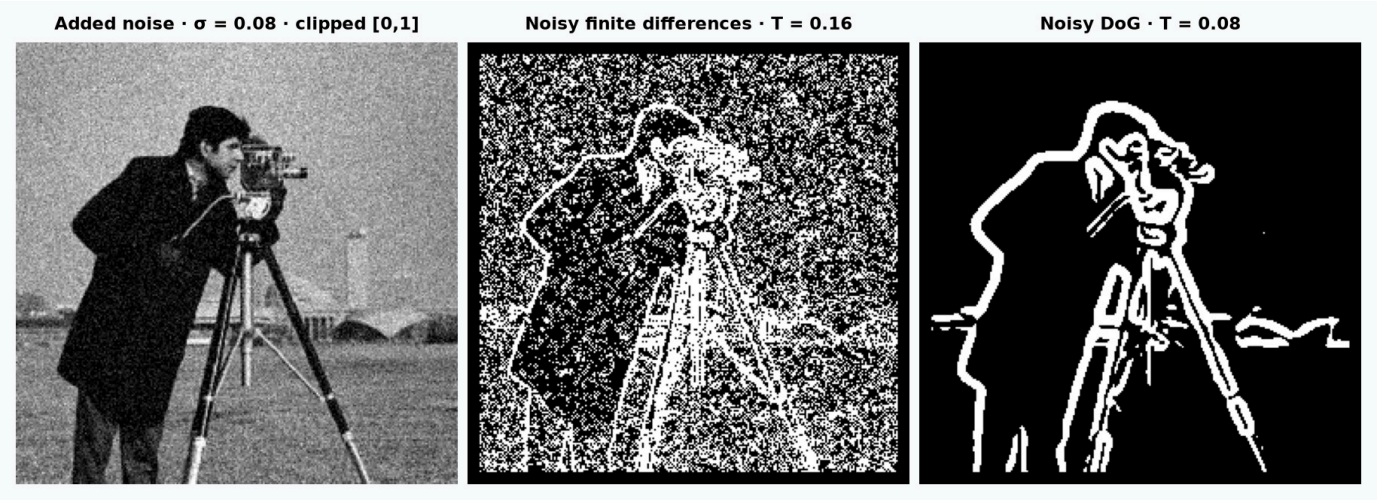
Measurement	Value
Two successive “same” convolutions vs. DoG · maximum error	0.307441
Two successive “same” convolutions vs. DoG · interior maximum error	8.95e-16
Full intermediate image vs. DoG · maximum error	8.95e-16

Two successive “same” convolutions crop the intermediate blur before differentiation. This removes samples outside the original frame and creates the boundary differences above. Keeping the full intermediate image and cropping once restores agreement across the complete output.

The edge maps use T = 0.08 for the Gaussian derivative and T = 0.16 for the original finite differences. Smoothing reduces response amplitudes, so these are separately selected visual settings, not a comparison of measured detection accuracy.

1.3 Sensitivity to added noise

Gaussian noise with σ = 0.08 is added using a fixed random seed, and the resulting intensities are clipped to [0, 1]. Each noisy gradient is compared with the same method applied to the clean image.



Cameraman image with added Gaussian noise and the resulting edge maps. Gaussian smoothing suppresses many of the noise responses visible with finite differences.

Interior vector-gradient residual

Measurement	Value
Finite-difference gradient residual RMS	0.152265
DoG gradient residual RMS	0.010304

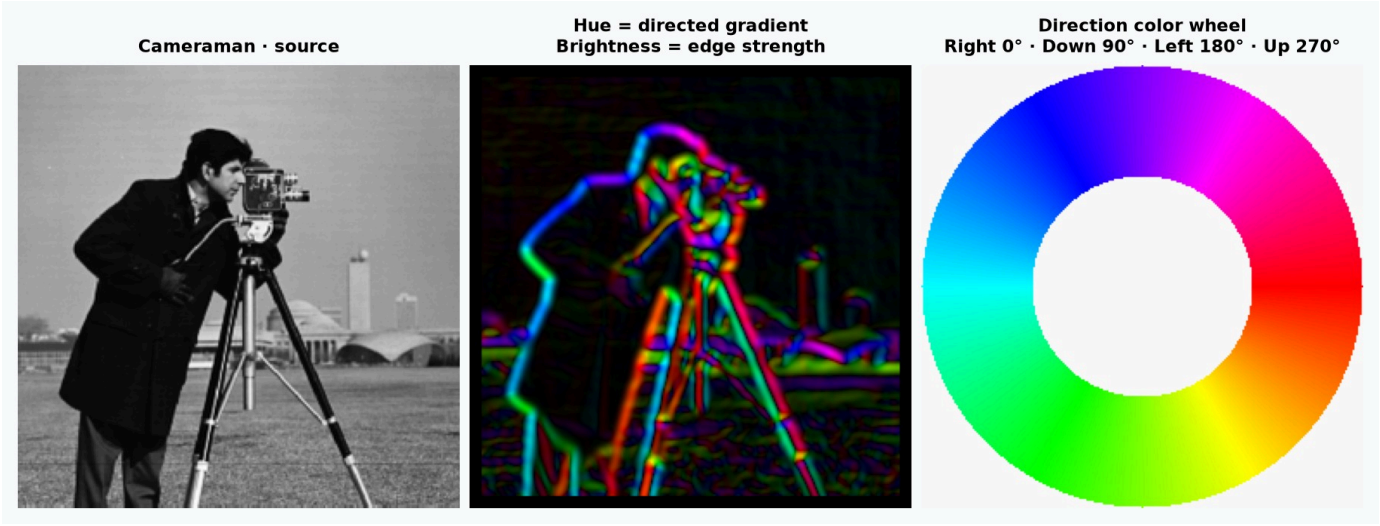
Effect of smoothing

The residual RMS decreases from 0.15226 for finite differences to 0.01030 for the derivative of Gaussian, a factor of 14.8. Smoothing suppresses many isolated noise responses, leaving a more continuous outline of the person and camera.

This test measures sensitivity to the added noise in this image. It does not measure recovery of detail lost through smoothing.

1.4 Gradient orientations

The Gaussian-derivative responses also define the direction of increasing intensity. Hue shows this direction, while brightness shows gradient strength.



Gradient orientation and direction color wheel. Hue represents direction in image coordinates; brightness represents edge strength. The seven-pixel boundary margin is excluded.

Reading the orientation map

The gradient is perpendicular to an edge tangent. Image rows increase downward, so 90° points down. Weak responses are dark, and the excluded seven-pixel boundary is black.

HSV mapping

$$h = (\theta \bmod 2\pi)/(2\pi) \quad s = 1$$
$$v = \min(|\nabla I| / 0.2386, 1)^{0.7}$$

The brightness reference, 0.2386, is the 99.5th percentile of interior gradient magnitude.

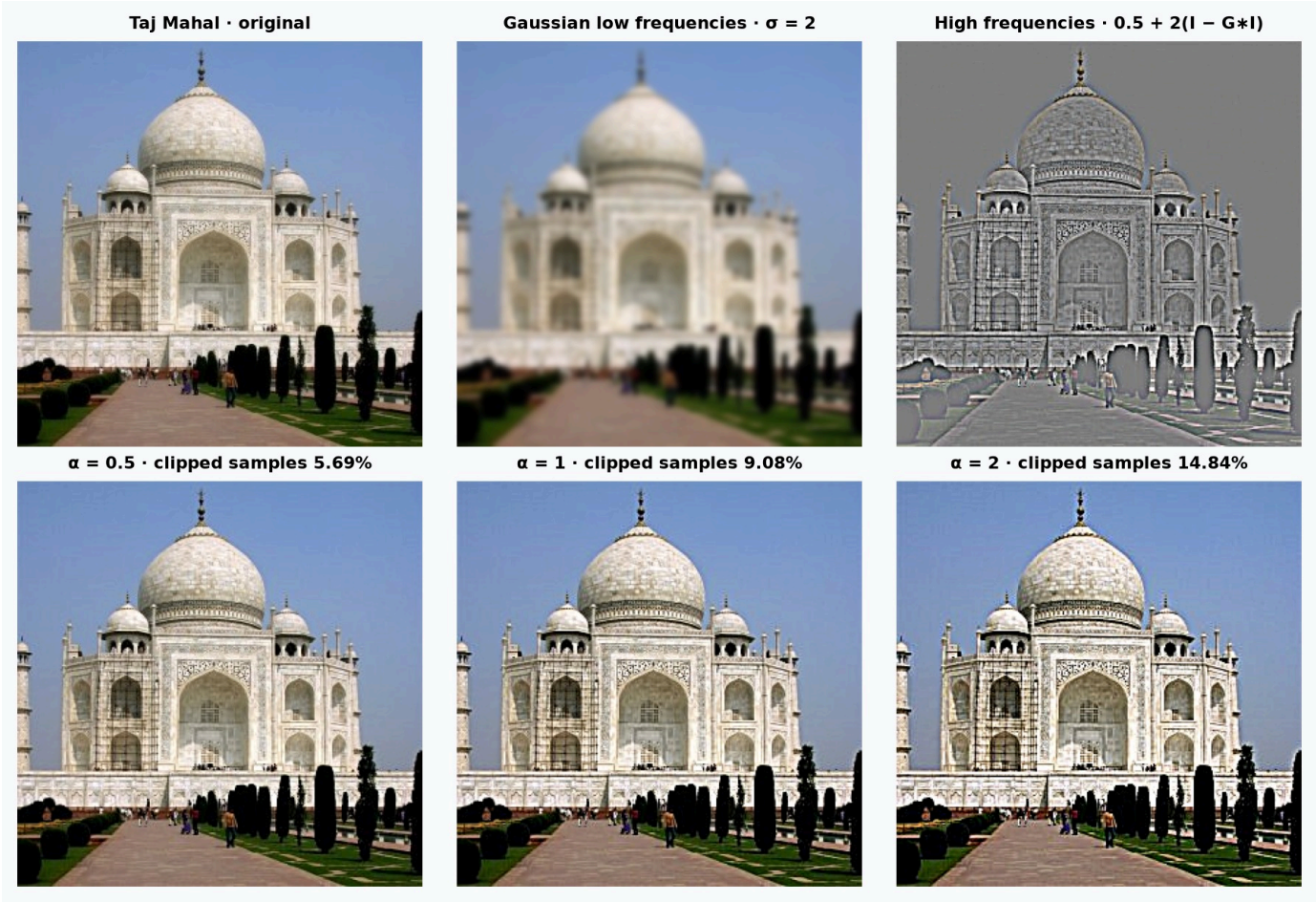
Computing the angle

The implementation uses an 18-term arithmetic series with range reduction and quadrant correction. It does not call arctangent, atan2, or an angle routine. Comparison with NumPy atan2, used only for validation, gives a maximum error of 4.44×10^{-16} radians. Appendix C gives the derivation and complete implementation.

2.1 Image sharpening

Unsharp masking adds a scaled high-frequency residual to the original image. The Gaussian blur supplies the low-frequency component; subtracting it from the input isolates detail.

Unsharp mask $B = G * I \quad H = I - B \quad S = I + \alpha H$



Taj Mahal: original image, Gaussian blur, high-frequency residual, and sharpening at α = 0.5, 1, and 2. The residual is displayed as 0.5 + 2H. Final display values are clipped to [0, 1].

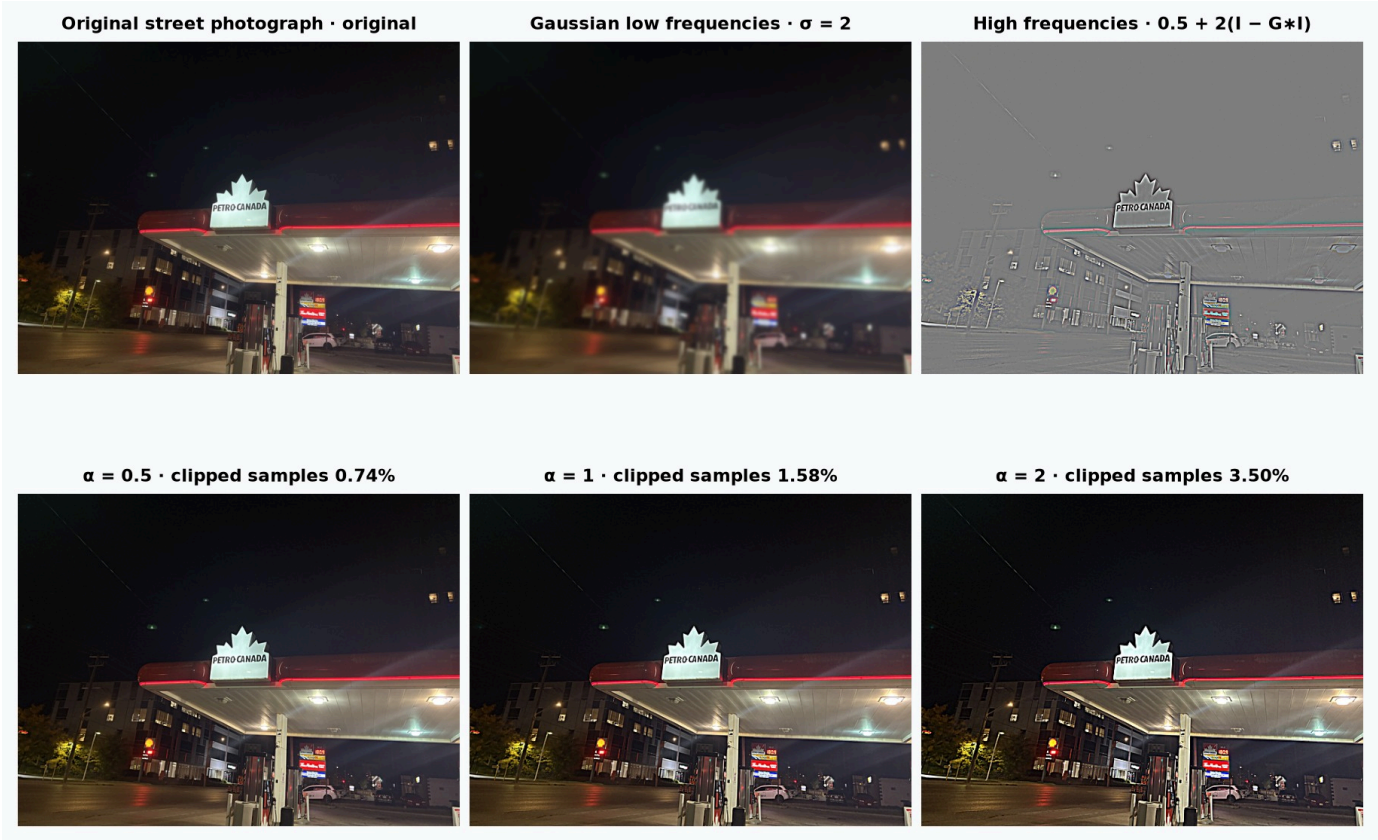
Effect of the sharpening amount

Increasing α strengthens the Taj Mahal's edges and texture. At the largest setting, halos and clipped highlights also become more apparent. Sharpening increases local contrast but can amplify noise and existing artifacts.

Both sharpening examples use a 13 × 13 Gaussian with σ = 2 pixels and symmetric boundaries. RGB channels are processed independently in float64. Residuals use the fixed display mapping 0.5 + 2H, so middle gray means zero.

2.1 Sharpening the street photograph

The same filter and sharpening amounts are applied to the original street photograph. Fine building edges and road texture become more distinct as α increases.



Original street photograph: original image, Gaussian blur, high-frequency residual, and sharpening at α = 0.5, 1, and 2. The residual is displayed as 0.5 + 2H. Final display values are clipped to [0, 1].

RGB values outside the display range

Image	α	Values outside [0, 1]
Taj Mahal	0.5	5.687%
Taj Mahal	1.0	9.075%
Taj Mahal	2.0	14.845%
Original street photograph	0.5	0.741%
Original street photograph	1.0	1.581%
Original street photograph	2.0	3.504%

The table counts scalar RGB values outside [0, 1] before export. The Taj Mahal image clips more heavily than the street image at every tested setting. Signed residuals and out-of-range values are retained during computation; only display exports are clipped.

2.1 One convolution and its limits

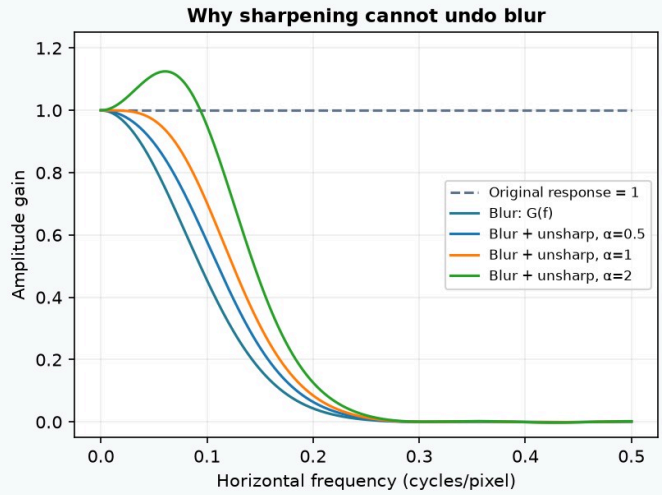
Expanding the unsharp-mask equation gives an equivalent single convolution. The unit impulse δ retains the original image, while the Gaussian term subtracts the blurred component.

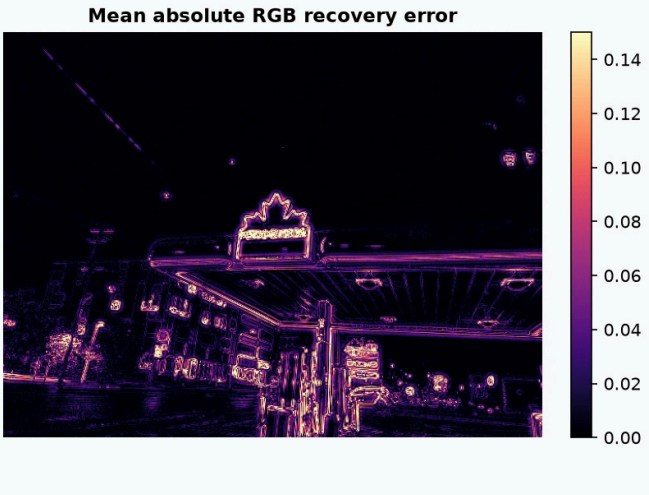
Sharpening kernel

$$S = [(1 + \alpha)\delta - \alpha G] * I$$
$$K = (1 + \alpha)\delta - \alpha G$$

The kernel sums to one and preserves a constant image when the boundary extension also preserves constants. Symmetric extension avoids the dark borders introduced by zero padding. At $\alpha = 1$, this formulation agrees with residual addition to a maximum error of 1.78×10^{-15} across both examples.

Why sharpening does not undo blur





Left: horizontal frequency response of the discrete Gaussian and the combined blur and sharpening operations. Right: mean absolute RGB error against the sharp reference at the selected setting.

Blur then sharpen

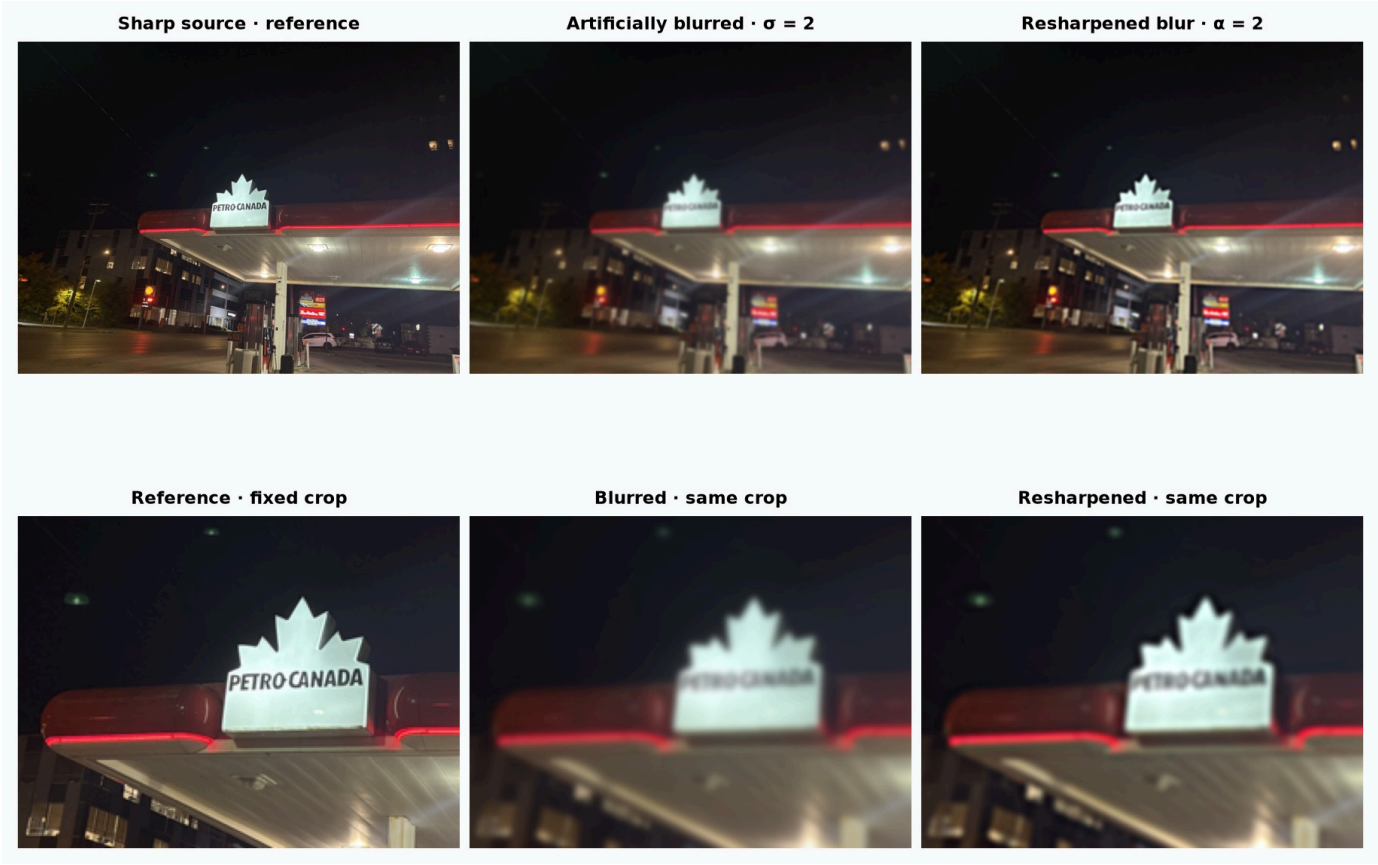
$$R(f) = G(f)[1 + \alpha - \alpha G(f)]$$

The combined frequency response generally differs from one. Frequencies strongly attenuated by the blur remain attenuated after finite amplification, and clipping can remove additional information. The nonzero error map shows that stronger edges do not imply complete recovery of the original detail.

The next comparison measures this loss against the original street photograph. Appendix D gives the single-convolution kernel code.

2.1 Blur and resharpen

The street photograph is first blurred, then sharpened with the same Gaussian. The original photograph provides a reference for the comparison.



Blur and sharpening of the original street photograph. The lower row shows the same fixed region from each image. α = 2 gives the lowest clipped RGB MSE among the three tested settings.

Recovery relative to the original street image · RGB reference in [0, 1]

Stage	α	Unclipped MSE	Unclipped PSNR (dB)	Clipped MSE	Clipped PSNR (dB)
Original	—	0	∞	0	∞
Blurred	—	0.001660	27.80	0.001660	27.80
Resharpened	0.5	0.001429	28.45	0.001429	28.45
Resharpened	1.0	0.001259	29.00	0.001259	29.00
Resharpened	2.0	0.001101	29.58	0.001099	29.59

Of the three tested amounts, α = 2 gives the lowest clipped RGB MSE: 0.001099, compared with 0.001660 after blurring. PSNR rises from 27.80 to 29.59 dB. This setting is selected against the original reference and applies to this experiment.

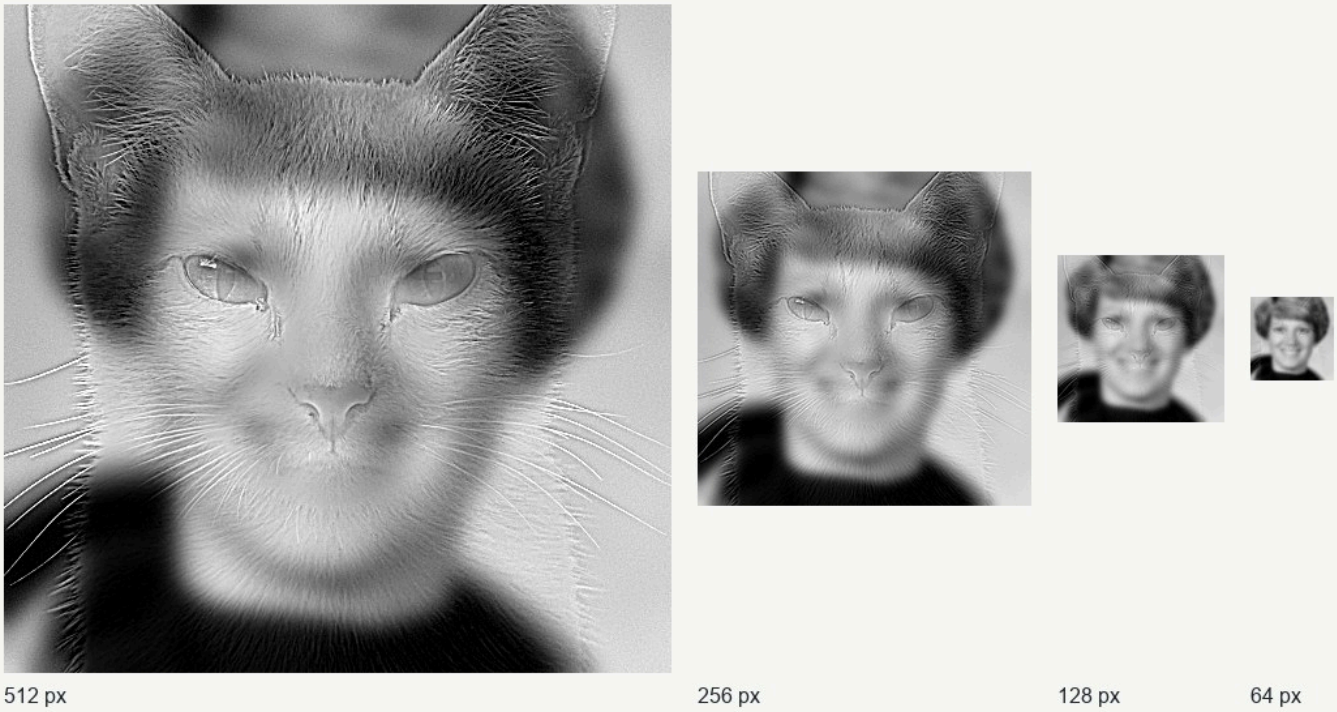
MSE is averaged over all RGB values. With a reference range of [0, 1], PSNR = 10 $\log_{10}(1/\text{MSE})$, in decibels. Both unclipped and display-clipped results are reported.

2.2 Hybrid images at different scales

A hybrid image combines the broad structure of one image with fine detail from another. Here, a blurred human portrait supplies the low frequencies and an aligned cat photograph supplies the high frequencies.

Hybrid construction $I_{hybrid} = G_{\sigma L} * A + g(B - G_{\sigma H} * B)$

A is the human portrait, B is the cat, g is the detail gain, and Gσ is a Gaussian kernel with standard deviation σ. The symbol ∗ denotes convolution.



Astronaut + portrait cat: the same hybrid at 512, 256, 128 and 64 pixels, reduced with Lanczos.

At 512 pixels, cat fur and eye outlines are prominent. At 64 pixels, the astronaut's eyes and smile become clearer. Both sources remain recognizable at 128 and 256 pixels, so this is a partial change in appearance with scale.

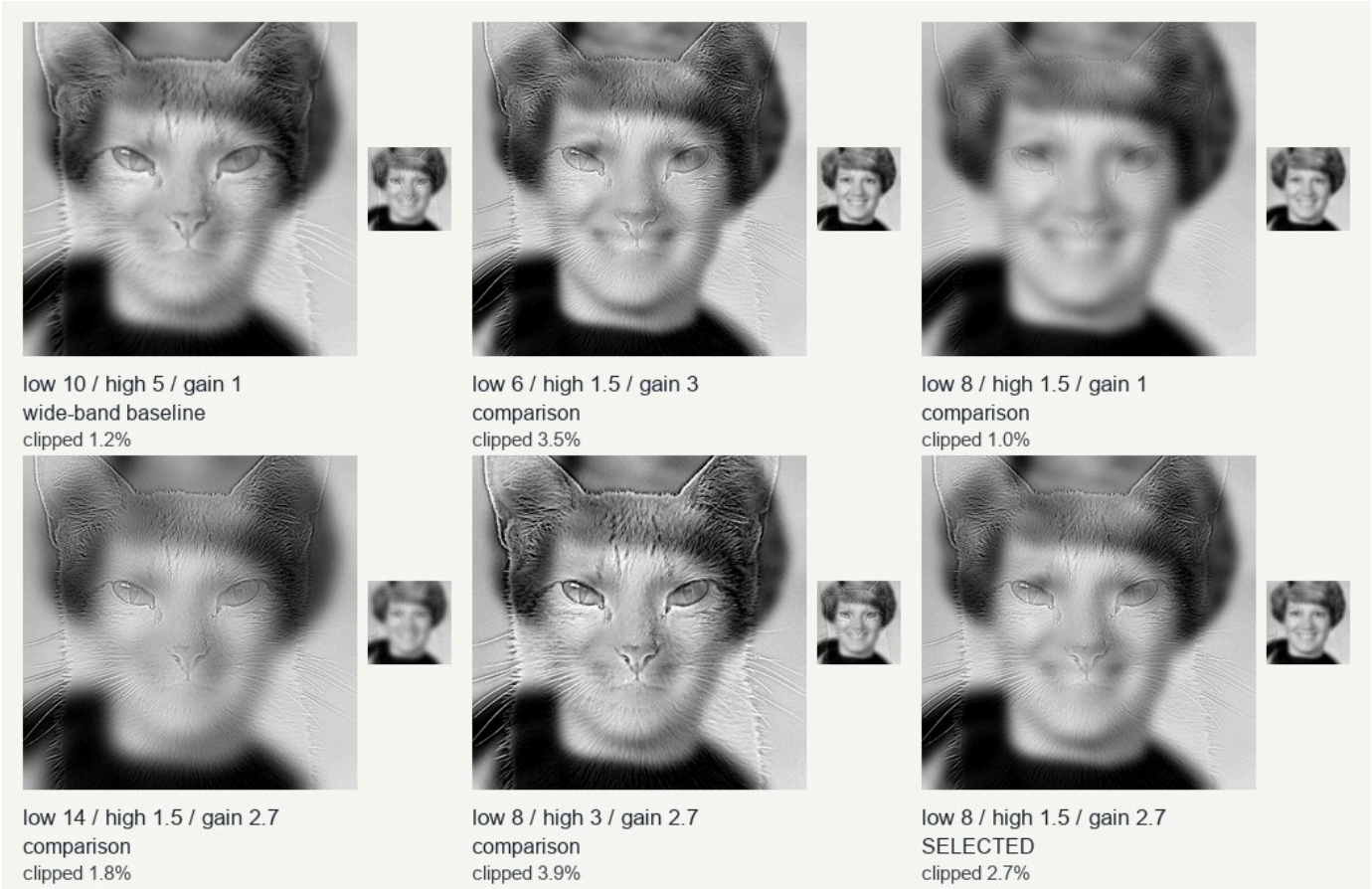
Selected settings at 512 x 512 pixels

Pair	Low-pass sigma (px)	High-pass sigma (px)	Gain	Values clipped
Derek + Nutmeg	8	1.5	2.25	2.36%
Astronaut + portrait cat	8	1.5	2.7	2.74%
Obama + tabby	6	1.5	2.2	2.07%

For a distance comparison, keep the largest image at a fixed display size and move farther away. The reduced versions provide another comparison; screen size, zoom and eyesight prevent assigning them a calibrated viewing distance.

2.2 Choosing filter widths and gain

A high-pass σ of 1.5 pixels removes more broad cat shading than σ = 5, leaving less cat structure in the reduced views. The selected gains retain visible fur detail.



Astronaut + portrait cat: filter-width and gain trials, synthesized at 512 x 512 and displayed at 256 and 64 pixels. The selected panel uses the final settings. Alignment remains fixed across the trials.

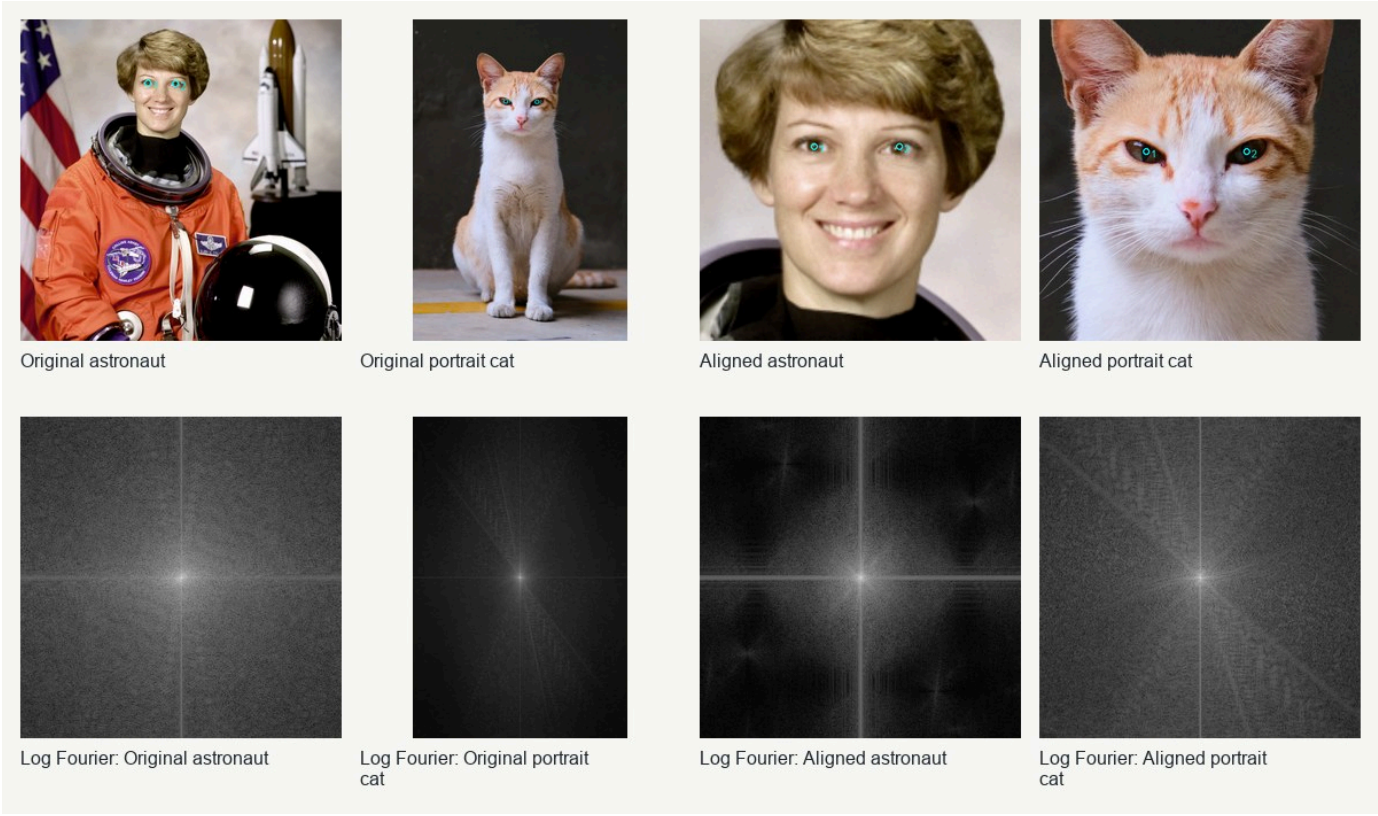
What changes in the trials

The six trials keep the selected alignment fixed. Increasing the astronaut's low-pass σ to 14 blurs the eyes and smile in the reduced view. Increasing the high-pass σ from 1.5 to 3 retains broader cat structure that competes with the human face.

The selected low-pass widths are 8 pixels for Derek and the astronaut, and 6 pixels for Obama. Their gains are 2.25, 2.7 and 2.2, respectively. These settings were chosen visually for these inputs. Supplementary comparisons vary alignment and filtering separately.

2.2 Aligning the two faces

The initial two-point similarity transforms map both pairs of eye centers to (170, 210) and (330, 210). Each transform combines rotation, uniform scale and translation. Exact eye alignment leaves the human mouth below the cat's muzzle in the Derek and astronaut pairs.



Original and aligned astronaut and portrait-cat inputs. Cyan circles show the alignment landmarks; spectra are calculated from the unmarked images. Both Fourier figures use the same logarithmic scale and display maximum.

Refining the human position

For those two pairs, the human image is scaled by 0.85 about (250, 210) and shifted upward by 8 pixels. Its eyes move to (182, 202) and (318, 202); the cat remains fixed. This reduces the extra mouth below the muzzle, although the pupils no longer coincide. Obama retains the common eye positions.

A 10% gain reduction from the reference setting limits clipping while retaining fine detail. The final clipped fraction is still slightly higher than for the larger human crop with exactly aligned eyes. The supplementary two-by-two comparisons show the geometry and filter changes separately.

Each original is resampled once with the composed similarity transform, using bilinear interpolation and replicated borders. Face proportions are unchanged. Original landmarks and exact 2 × 3 matrices are saved with the experiment records.

2.2 Frequency content of the hybrid

A Gaussian has no sharp cutoff. Its approximate half-amplitude frequency provides a reference for the selected filter widths.

Half-amplitude frequency $f_{1/2} = \sqrt{(2 \ln 2)} / (2\pi\sigma)$

For the astronaut pair, the low-pass and complementary high-pass values are 0.0234 and 0.1249 cycles per output pixel, respectively. Filtering uses spatial convolution; Fourier plots show the resulting frequency content.



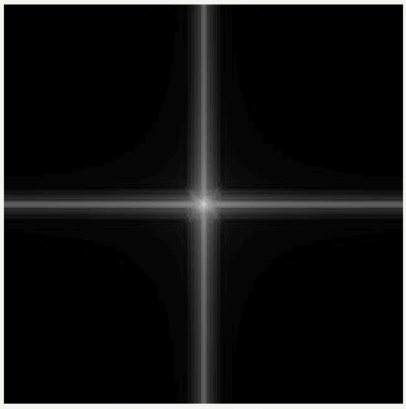
Gaussian low pass



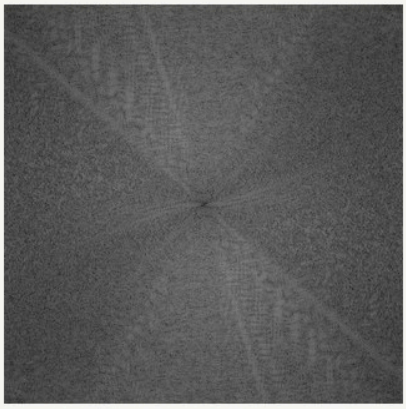
Gain-scaled signed high pass



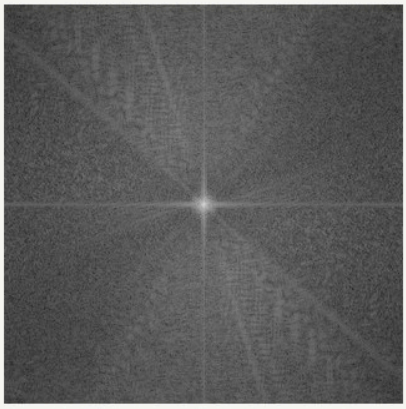
Final hybrid



Log Fourier: Gaussian low pass



Log Fourier: Gain-scaled signed high pass



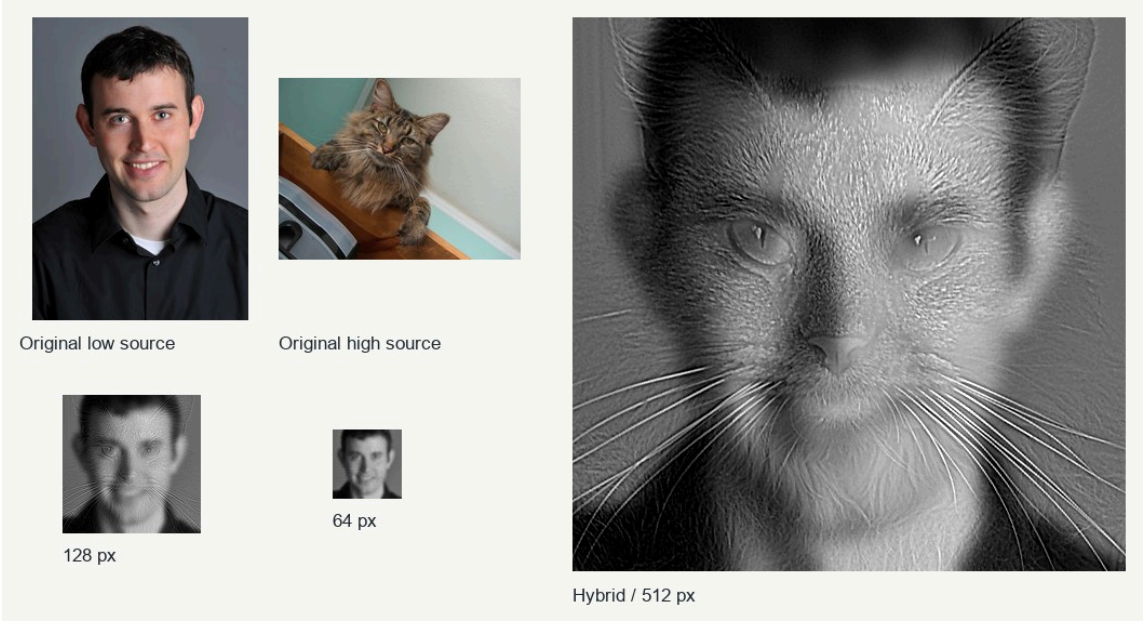
Log Fourier: Final hybrid

Low-pass energy is concentrated near the center; the high-pass suppresses central energy and retains fine detail. Adding the two components produces the hybrid. Both Fourier figures use the same logarithmic scale and display maximum.

Fourier display $\log(1 + 1{,}000{,}000\ |\text{fftshift}(\text{fft2}(Y))| / N)$

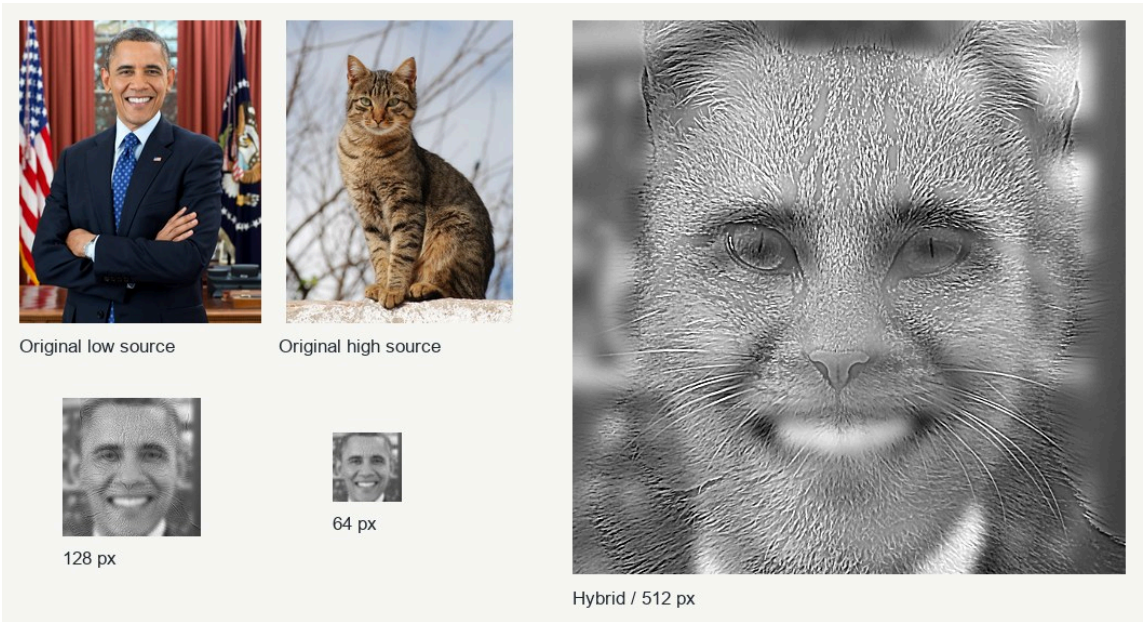
Y is luminance and N is the number of image pixels. Division by N permits comparison of differently sized inputs. The source and process spectra share one display maximum. No window or mean subtraction is applied.

2.2 Two more portrait–cat pairs



Derek + Nutmeg: original sources and the final hybrid at 512, 128 and 64 pixels.

The adjusted human scale and position reduce the extra mouth below the cat muzzle. Fur and eye outlines remain visible at 512 pixels; Derek's facial structure becomes clearer at 64 pixels. Intermediate sizes retain features from both sources.



Obama + tabby: original sources and the final hybrid at 512, 128 and 64 pixels.

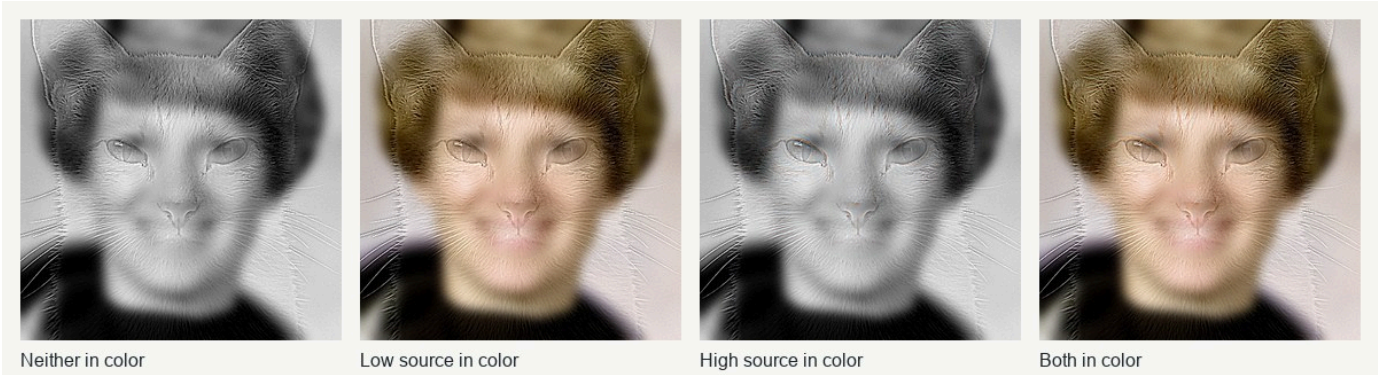
Gain 2.2 retains fine tabby fur; higher gain clips more bright fur pixels. Obama's smile becomes clearer at reduced size, while whiskers remain visible at intermediate sizes.

Supplementary figures retain each pair's registration, components and parameter comparisons, plus the Earth + Moon limitation study. Its similar outlines and overlapping surface structure do not produce a clear change in perceived subject.

Additional parameter trials and diagnostic figures are available in the online supplement.

2.2 Color in the frequency bands

Four variants retain color in neither source, only the low-frequency source, only the high-frequency source, or both. Alignment, filter widths and gain remain identical.



Four color variants with identical alignment, filter widths and gain. Luminance conversion uses weights 0.2126, 0.7152 and 0.0722. Color in the low-frequency source preserves broad skin and hair regions; color in the high-frequency source remains mainly around cat edges and fur.

How color changes the result

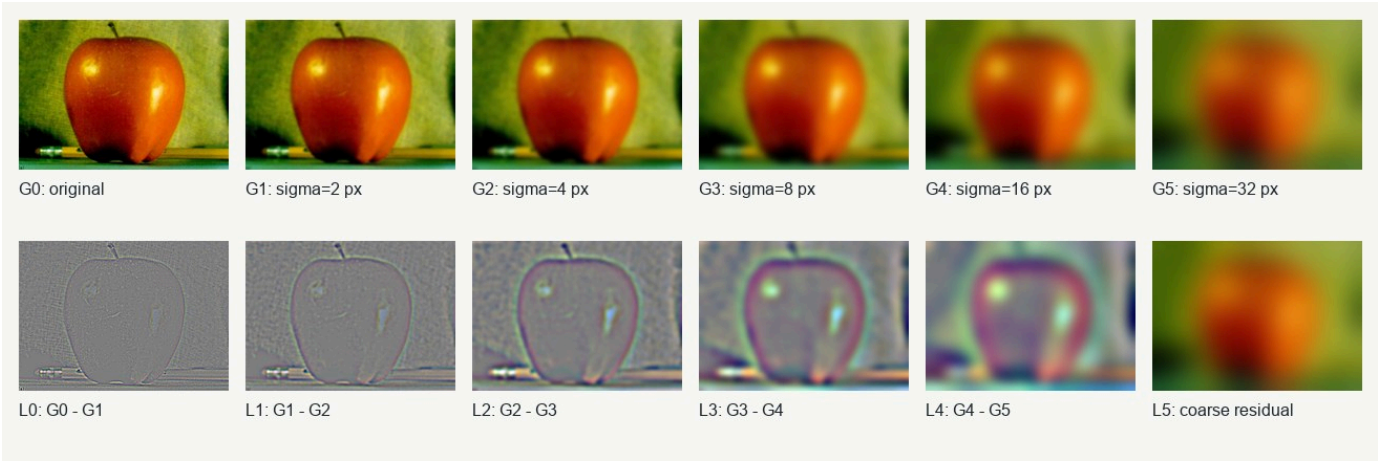
Low-frequency color preserves the astronaut's broad skin and hair regions, agreeing with the face visible at smaller sizes. Grayscale fur remains visible at larger sizes. High-frequency color remains mainly around fine cat edges because filtering removes the broad coat color.

Keeping color in both sources adds competing hues around some cat edges. The main hybrid examples use grayscale so the visible subject can be assessed without this broad color difference.

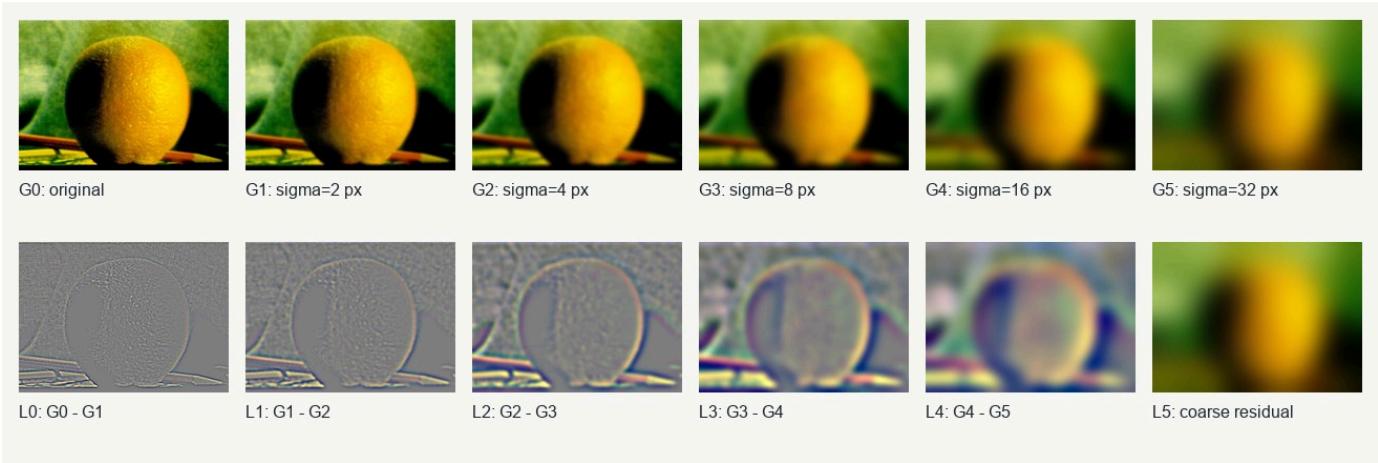
2.3 Gaussian and Laplacian stacks

The supplied burt_apple.png and burt_orange.png images are each 740 × 530 pixels. One resize with area interpolation gives 512 × 367 working images, reducing computation while preserving aspect ratio to the nearest output pixel.

Every Gaussian-stack level keeps the same dimensions. G_0 is the original; G_1 through G_5 independently blur it at σ = 2, 4, 8, 16 and 32 pixels. Each width therefore refers to the original image. Construction is explicit, with no downsampling or built-in pyramid routine.



Apple: Gaussian stack above and Laplacian stack below, including the coarse residual. Every level retains the working resolution of 512 × 367 pixels. Signed detail bands use 0.5 + 2.5L for display; the residual uses its actual intensity.



Orange: Gaussian stack above and Laplacian stack below, including the coarse residual. Every level retains the working resolution of 512 × 367 pixels. Signed detail bands use 0.5 + 2.5L for display; the residual uses its actual intensity.

Laplacian detail and residual $L_i = G_i - G_{i+1}$ $(i = 0, \ldots, 4)$; $L_5 = G_5$

The five detail bands sum to the original minus G_5. Adding the coarse residual restores large-scale brightness and color, reconstructing the image up to floating-point rounding.

2.3 Reconstructing the oraple

The selected levels show how each source contributes at different scales. The final row sums all five detail bands and the coarse residual.

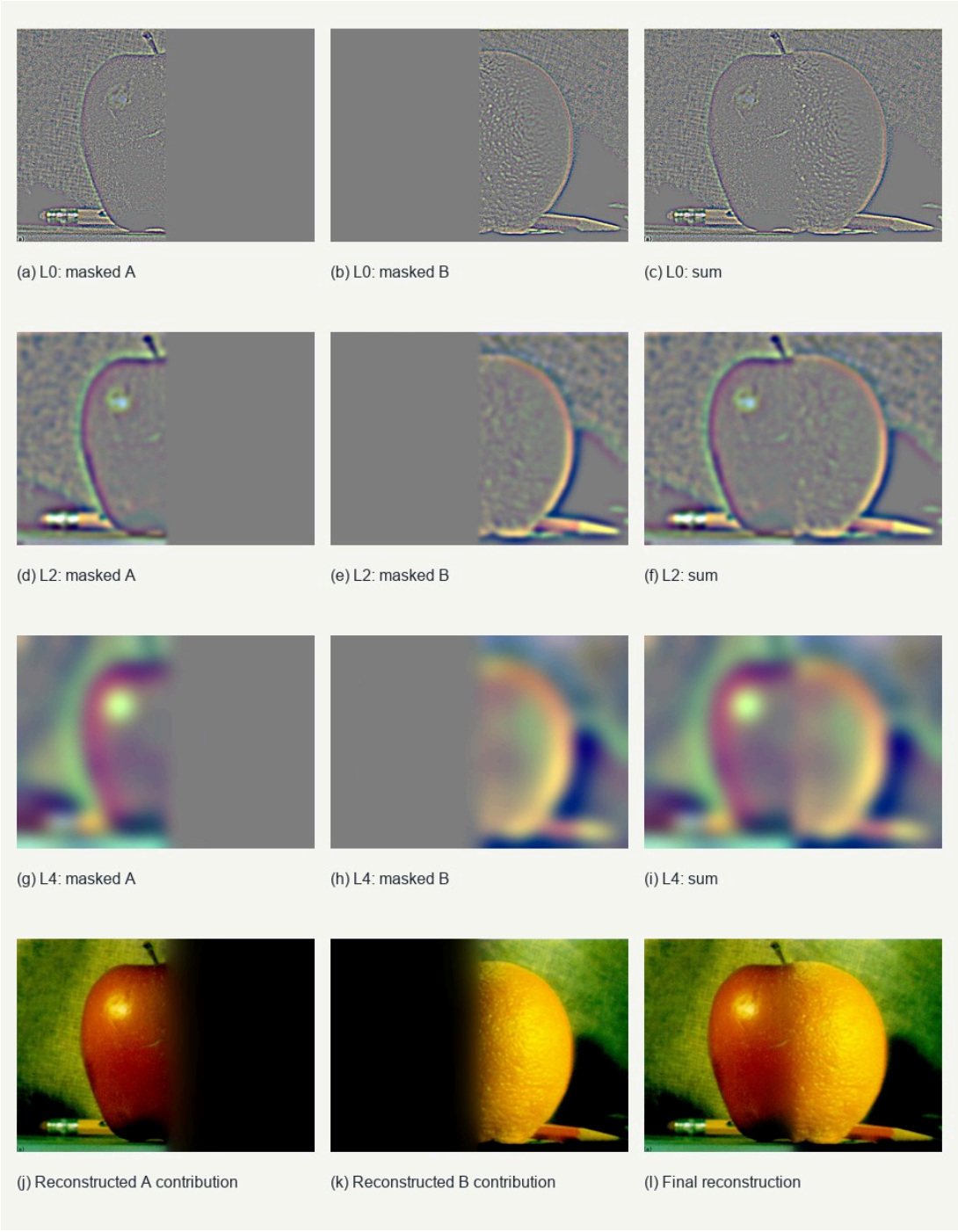


Figure 3.42 (a)–(l) layout: rows 1–3 show levels 0, 2, and 4; columns show the masked apple, masked orange, and their sum. Row 4 sums all five detail bands and the coarse residual to reconstruct the two contributions and final oraple. Detail bands share a symmetric display scale within each row; reconstructed images use actual [0, 1] intensity.

2.4 A transition that changes with scale

A binary mask selects apple on the left and orange on the right. Its Gaussian stack uses the same smoothing scales as the image stacks.

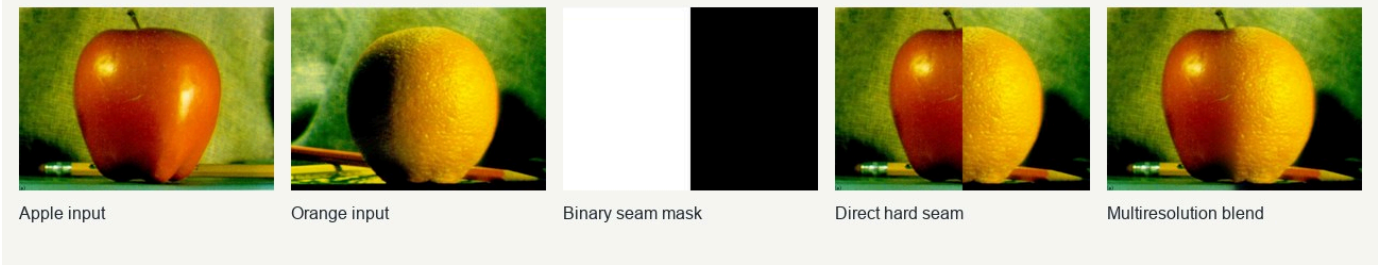


Gaussian stack of the left/right selection mask, using the same scales as the image stacks. The transition widens at coarser levels. White selects apple and black selects orange.

Blend each band, then sum

$$S_i = M_i L_{A,i} + (1 - M_i) L_{B,i}; \quad I_{blend} = \Sigma_{i=0}^{5} S_i$$

The rule includes the coarse residual. The finest mask has a sharp transition; coarser masks spread brightness and color changes across wider regions while retaining local texture.



Supplied inputs, selection mask, direct binary cut, and multiresolution oraple. Both methods use the same 512 × 367 images with preserved aspect ratio. The seam lies between zero-based columns 255 and 256.

Both methods use the same inputs and initial mask. The direct cut leaves a visible vertical change in brightness and color. The mean absolute RGB difference across adjoining seam columns falls from 0.1716 to 0.0279 with multiresolution blending. This measures the local discontinuity, not overall perceptual quality.

Reconstruction accuracy

Comparison	Maximum absolute error
Apple reconstruction	1.110e-16
Orange reconstruction	2.220e-16
Final image vs. sum of mixed bands	3.331e-16

The seam lies between zero-based columns 255 and 256. Measurement uses the floating-point reconstruction before clipping. Stack construction and reconstruction remain unclipped; display export limits 2.35% of scalar RGB values to [0, 1].

2.4 A cat face inside another coat

The tabby's central face is placed within the orange-and-white cat's ears, outer whiskers and chest. A 24-vertex polygon follows the crown, temples, cheeks and chin, keeping forehead stripes inside the selection. A similarity transform aligns the eye centers with uniform scale, rotation and translation.



Cat face blending: original and registered photographs. Cyan circles mark the eye centers and their common output positions. Tabby photo: Joaquim Alves Gaspar (Alvesgaspar), CC BY-SA 3.0. Orange-and-white cat: Himanshu Choudhary, Unsplash License. The adapted composite and tabby-derived figures are distributed under CC BY-SA 3.0.

A common comparison

Both custom examples use complete registered photographs, sampled with bilinear interpolation and replicated borders. Exposure, color and backgrounds are unchanged; no retouching or background transfer precedes blending.

The hard cut, single-scale feathering and multiresolution blend use identical registered arrays and the same binary selection. Feathering applies the exact coarsest Gaussian mask to the full images. Multiresolution blending applies the corresponding Gaussian mask to each Laplacian band.

Selection weights

The cat uses five detail bands at σ = 2, 4, 8, 16 and 32 pixels, plus the coarse residual. An initial mask σ of 8 softens the crown and cheeks while keeping the eyes, nose and muzzle within the tabby selection.

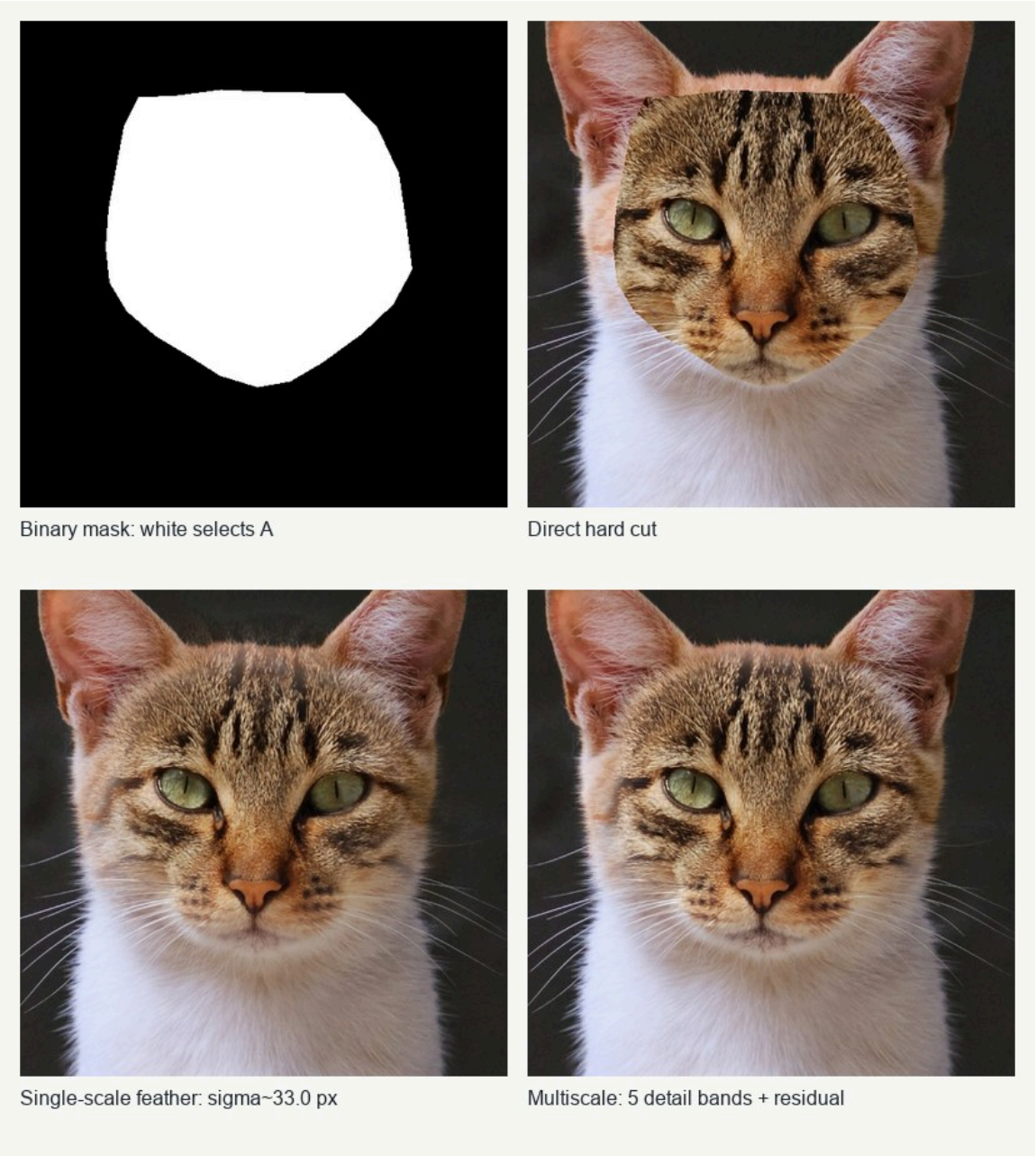
Nominal mask width after further smoothing

$$\sigma_{mask} = \sqrt{(8^2 + \sigma_{stack}^2)}$$

The resulting cat mask widths are 8, 8.2, 8.9, 11.3, 17.9 and 33.0 pixels. These are nominal equivalent widths; weights come from the discrete convolutions. Only selection weights change; photographs and image detail bands stay fixed. The oraple retains a binary finest mask.

2.4 Cat blending: boundary and texture

The direct cut exposes the polygon around the face. Broad feathering mixes the two coat textures over one wide transition.

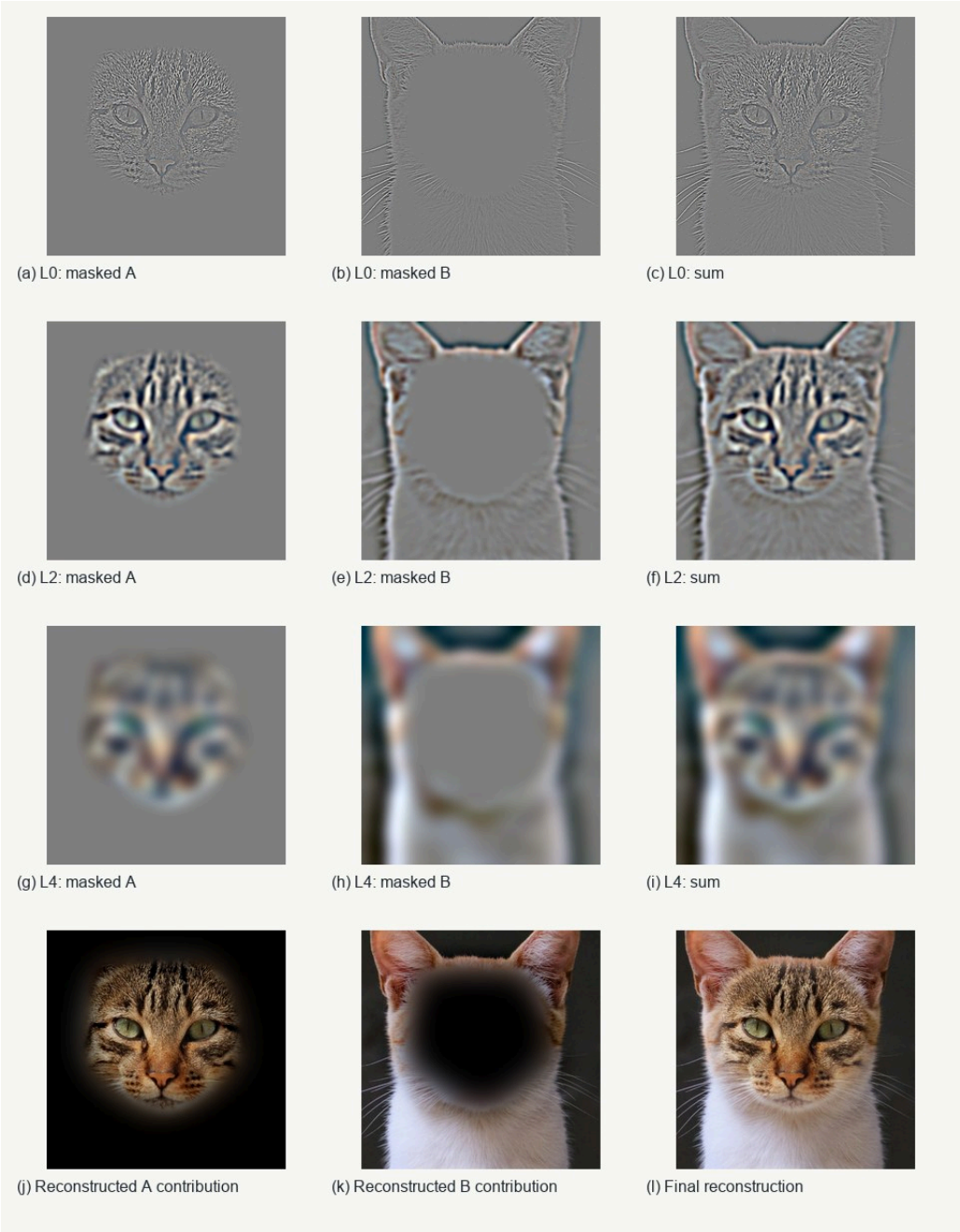


Three methods with the same registered inputs and binary selection. The mask is initially smoothed at σ = 8 pixels before constructing the mask stack.

Multiresolution blending reduces the coat-color change and retains clearer tabby detail around the cheek and muzzle. Eye outlines, forehead stripes, nose and muzzle remain distinct. The gradual transition still contains differences in muzzle shape and fur direction; blending does not correct anatomy.

2.4 Cat contributions at each scale

A maximum image σ of 16 leaves a more visible facial patch. σ = 32 softens the coat boundary; σ = 64 spreads the color transition farther into the surrounding fur without a clear improvement in the fur boundary.

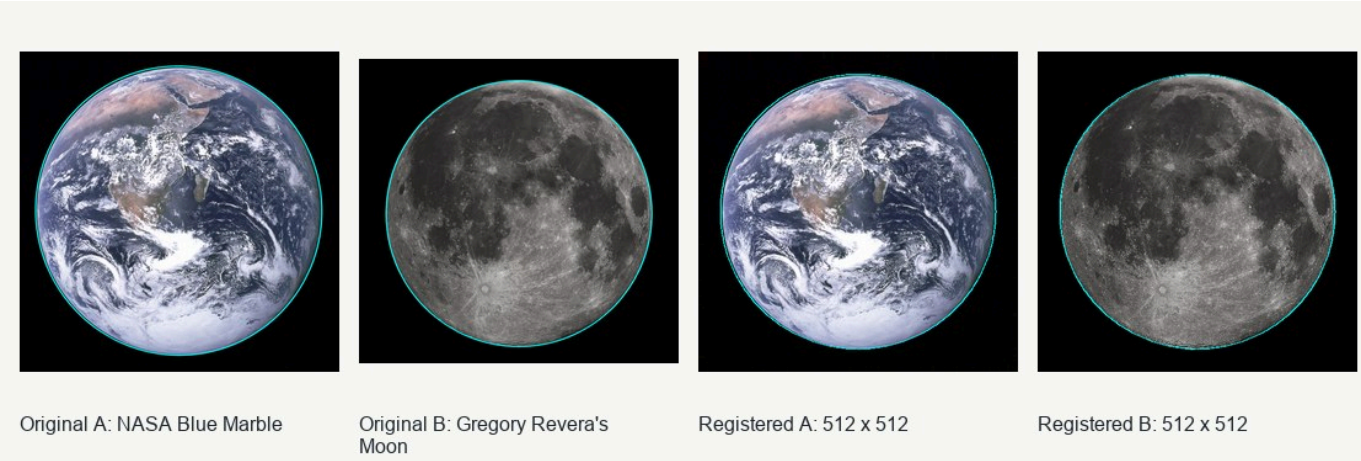


Cat face blending: rows 1–3 show masked A, masked B and their sum at Laplacian levels 0, 2 and 4. The final row reconstructs each contribution and the result using all detail bands plus the coarse residual, following Burt–Adelson Figure 10. Each detail row uses one shared symmetric contrast scale; reconstructed rows retain actual intensity.

Supplementary trials compare four, five and six detail bands with other inputs fixed. Initial-mask trials compare σ = 0, 2, 4, 8, 12 and 16 pixels: a binary mask leaves an abrupt crown texture change, σ = 4 leaves a more visible fur boundary, and σ = 12 or 16 mixes a wider region of the two coats. The selected σ = 8 balances these effects.

2.4 Joining Earth and Moon

An S-shaped boundary joins Earth's oceans and clouds to lunar craters. The similar disk outlines support this composite, although the same photographs did not produce a clear subject change in the hybrid experiment.



Earth-Moon blending: original and registered photographs. Cyan outlines mark the fitted source ellipses and target circle. The larger axis scale factor exceeds the smaller by 1.06% for Earth and 0.03% for the Moon. Earth: NASA/Apollo 17, public domain. Moon: Gregory H. Revera, CC BY-SA 3.0. The adapted composite and Moon-derived figures are distributed under CC BY-SA 3.0.

The largest source contours above luminance 0.04 are fitted with cv2.fitEllipse and mapped affinely to a circle of radius 220 pixels at (256, 256). Thresholding measures the outlines only; the transforms sample original RGB pixels. Earth's axis scale factors differ by 1.06%; the Moon's differ by 0.03%.

Earth–Moon uses six detail bands, adding image $\sigma = 64$ pixels, plus the coarse residual. The initial mask again uses $\sigma = 8$, giving one further nominal mask width of 64.5 pixels.

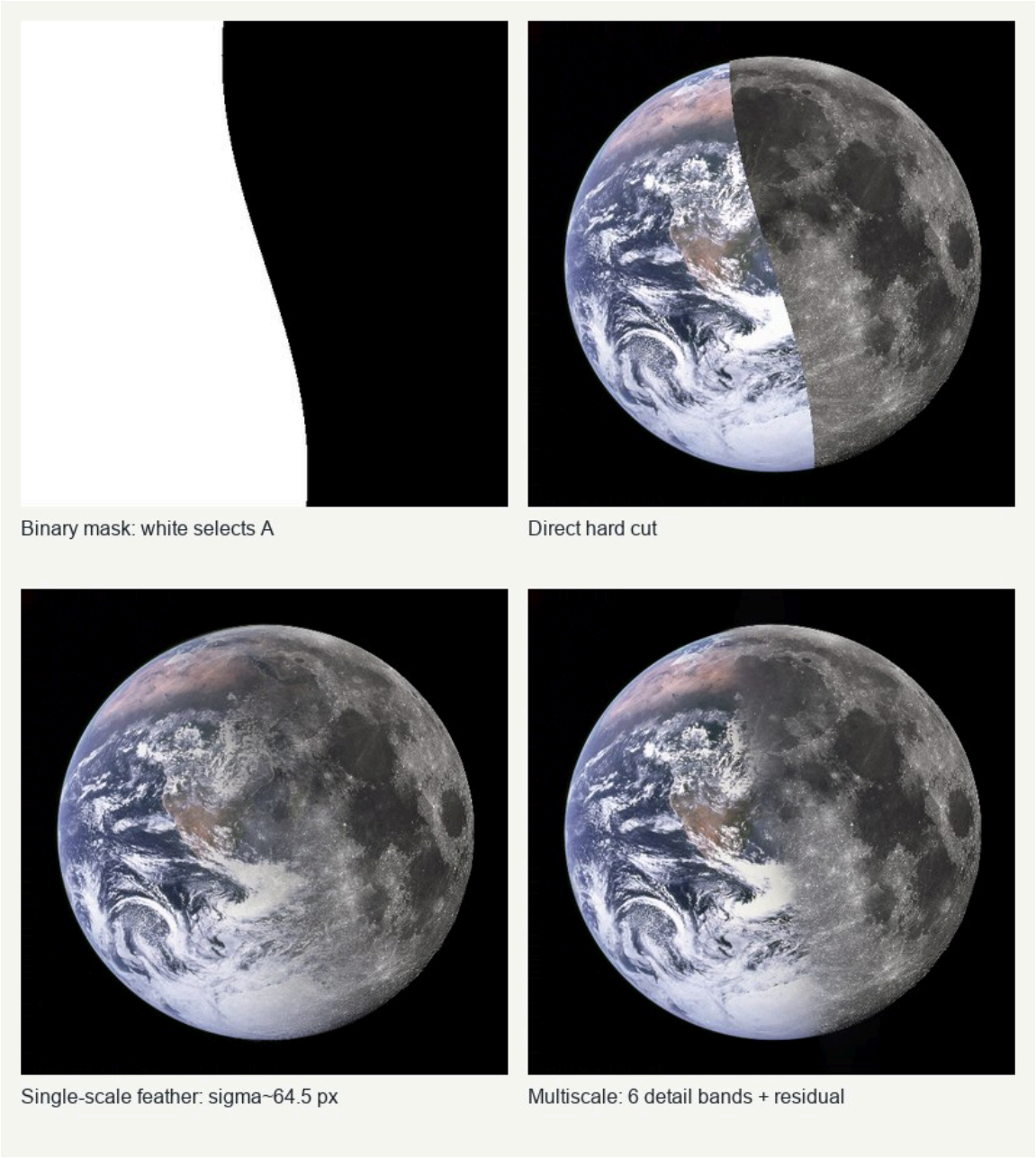
Blend settings and mean boundary differences

Parameter or measurement	Cat face blending	Earth-Moon blending
Selection mask	polygon	sinusoidal
Detail bands	5	6
Maximum image sigma (px)	32	64
Boundary jump: hard cut	0.2089	0.2718
Boundary jump: feathering	0.0289	0.0422
Boundary jump: multiresolution	0.0304	0.0424
Scalar values clipped	0.00%	4.92%

Boundary jump is the mean absolute RGB difference between adjacent pixels on opposite sides of the original binary contour. It includes natural texture as well as the seam. Feathering mixes more detail and gives smaller values; this alone does not establish a better-looking or physically realistic composite. Processing retains signed float64 values; only final display images are clipped to [0, 1].

2.4 Planet blending: brightness and detail

The Earth–Moon blend uses a maximum image σ of 64 pixels because the brightness mismatch extends across much of the disk. Fewer bands leave a more visible boundary.



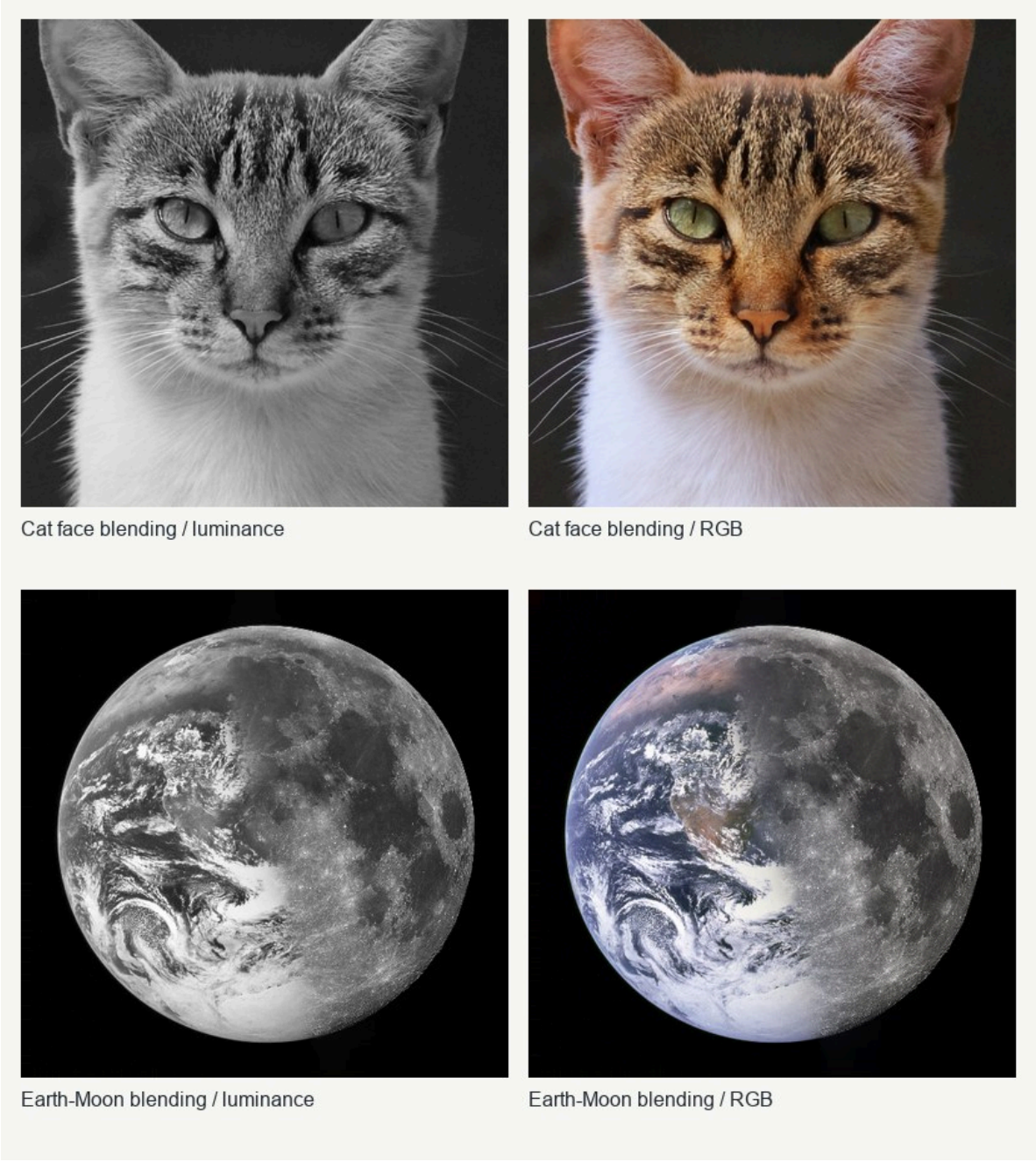
Three methods with the same registered inputs and binary selection. Initial mask smoothing at σ = 8 pixels reduces the abrupt contrast change where the selection meets the disk outline.

Multiresolution blending reduces the brightness step and keeps cloud and crater detail more localized than broad feathering. A binary finest mask leaves an abrupt outer-edge join; wider masks combine texture across a larger region. The same six initial widths and four-, five- and six-band comparisons are retained in the supplementary figures.

The composite remains an imaginary surface: lunar features do not continue Earth's clouds. Smoothing a boundary cannot reconcile different surface structures.

2.4 Color separates the source regions

The final composites use RGB. The tabby's coat separates its face from white fur and orange ears; blue oceans distinguish Earth from the nearly neutral Moon. These regions are less distinct in grayscale, even though the boundary is still smoothed.



Luminance and RGB results with the same registration, masks and stack settings. Luminance is 0.2126R + 0.7152G + 0.0722B; RGB channels are blended independently.

Registration, masks and stack settings are identical in this comparison. RGB channels are blended independently. Supplementary figures retain the matched boundary crops, complete mask stacks, initial-mask sweeps and band-count comparisons for both examples.

Additional parameter trials and diagnostic figures are available in the online supplement.

IMPLEMENTATION APPENDIX

A Convolution implementation

The shared helper validates the inputs, reverses the kernel, and pads the image with zeros. Both implementations return the same image shape and use the same padding convention.

Input validation, kernel reversal, and padding

```
def _convolution_inputs(image, kernel):
    image = np.asarray(image, dtype=np.float64)
    kernel = np.asarray(kernel, dtype=np.float64)
    if image.ndim != 2 or kernel.ndim != 2:
        raise ValueError('image and kernel must both be two dimensional')
    if not image.size or not kernel.size:
        raise ValueError('image and kernel must be nonempty')
    kh, kw = kernel.shape
    padded = np.pad(image, ((kh // 2, (kh - 1) // 2), (kw // 2, (kw - 1) // 2)),
mode='constant')
    return (image, kernel[::-1, ::-1], padded)
```

Four-loop convolution

```
def convolve_four_loops(image, kernel):
    image, flipped, padded = _convolution_inputs(image, kernel)
    height, width = image.shape
    kh, kw = flipped.shape
    out = np.zeros_like(image)
    for y in range(height):
        for x in range(width):
            for ky in range(kh):
                for kx in range(kw):
                    out[y, x] += flipped[ky, kx] * padded[y + ky, x + kx]
    return out
```

IMPLEMENTATION APPENDIX

B Vectorization and image gradients

The two-loop version accumulates a complete shifted image slice for each kernel element. This changes the Python loop structure without changing the convolution sum.

Two-loop convolution

```
def convolve_two_loops(image, kernel):
    image, flipped, padded = _convolution_inputs(image, kernel)
    height, width = image.shape
    kh, kw = flipped.shape
    out = np.zeros_like(image)
    for ky in range(kh):
        for kx in range(kw):
            out += flipped[ky, kx] * padded[ky:ky + height, kx:kx + width]
    return out
```

Gradient and edge magnitude

```
gx = convolve_two_loops(image, DX)
gy = convolve_two_loops(image, DY)
magnitude = np.sqrt(gx * gx + gy * gy)
edges = magnitude > threshold
```

Cameraman input preparation

The supplied image has a thin white border. The crop (1, 7, 535, 541), in (left, top, right, bottom) coordinates with the right and bottom bounds excluded, removes it. The square crop is resized to 256 × 256 pixels with Lanczos interpolation.

Calculations use floating-point intensities in [0, 1] with no further normalization before thresholding. The seven-pixel evaluation margin covers the largest Gaussian-derivative support used in Part 1.

IMPLEMENTATION APPENDIX

C Orientation without angle routines

For nonzero gradients, the ratio below lies in [0, 1]. When it exceeds $\sqrt{2}-1$, range reduction bounds the series argument by $\sqrt{2}-1$. Eighteen terms approximate arctangent; component swaps and signs recover the quadrant.

Range reduction

$$r = \min(|g_y|, |g_x|) \,/\, \max(|g_y|, |g_x|)$$

$$\operatorname{atan}(r) = \pi/4 + \operatorname{atan}((r-1)/(r+1)) \quad \text{for } r > \sqrt{2}-1$$

$$\operatorname{atan}(z) \approx z - z^3/3 + z^5/5 - \dots$$

Arithmetic-only orientation implementation

```
def orientation_without_angles(gx, gy, terms=18):
    gx, gy = np.broadcast_arrays(np.asarray(gx, float), np.asarray(gy, float))
    if not np.all(np.isfinite(gx)) or not np.all(np.isfinite(gy)):
        raise ValueError('gradient components must be finite')
    if terms < 1:
        raise ValueError('terms must be positive')
    ax, ay = (np.abs(gx), np.abs(gy))
    high, low = (np.maximum(ax, ay), np.minimum(ax, ay))
    ratio = np.divide(low, high, out=np.zeros_like(high), where=high != 0)
    reduce = ratio > np.sqrt(2.0) - 1.0
    z = np.where(reduce, (ratio - 1.0) / (ratio + 1.0), ratio)
    z2 = z * z
    poly = np.zeros_like(z)
    for n in range(terms - 1, -1, -1):
        poly = (-1.0) ** n / (2 * n + 1) + z2 * poly
    small_angle = z * poly + np.where(reduce, np.pi / 4, 0.0)
    angle = np.where(ay > ax, np.pi / 2 - small_angle, small_angle)
    angle = np.where(gx < 0, np.pi - angle, angle)
    angle = np.where(gy < 0, -angle, angle)
    return np.where(high == 0, 0.0, angle)
```

A zero gradient is assigned orientation zero and displayed black. NumPy atan2 is used only as an independent validation reference.

D Gaussian derivatives and sharpening

Full convolution retains the complete Gaussian-derivative support. The sharpening kernel combines the identity and Gaussian terms into one image convolution.

Build Gaussian derivative kernels

```
g1 = cv2.getGaussianKernel(13, 2.0)
G = g1 @ g1.T
Kx = convolve2d(G, DX, mode='full')
Ky = convolve2d(G, DY, mode='full')
gx = convolve2d(image, Kx, mode='same', boundary='fill')
gy = convolve2d(image, Ky, mode='same', boundary='fill')
```

Single-convolution sharpening kernel

```
def unsharp_kernel(gaussian, alpha):
    gaussian = np.asarray(gaussian, float)
    if gaussian.ndim != 2 or any((n % 2 == 0 for n in gaussian.shape)):
        raise ValueError('Gaussian must have odd dimensions')
    if not np.isclose(gaussian.sum(), 1.0):
        raise ValueError('Gaussian must sum to one')
    kernel = -float(alpha) * gaussian.copy()
    kernel[gaussian.shape[0] // 2, gaussian.shape[1] // 2] += 1 + float(alpha)
    return kernel
```

The sharpening kernel is convolved with each RGB channel using “same” output and symmetric boundary extension. All intermediate arrays use float64; clipping is deferred until display export.

E Hybrids and multiresolution stacks

The hybrid combines a low-pass source with a scaled high-pass source. The stack construction keeps every level at the original resolution; the last Gaussian level is retained as the coarse residual.

Hybrid synthesis

```
low = gaussian(A, sigma_low)
high = B − gaussian(B, sigma_high)
hybrid = low + gain ∗ high
```

Stack construction and reconstruction

```
G = [I] + [gaussian(I, 2 ∗ 2 ** i) for i in range(5)]
L = [G[i] − G[i + 1] for i in range(5)] + [G[−1]]
S = [M[i] ∗ LA[i] + (1 − M[i]) ∗ LB[i] for i in range(6)]
result = sum(S)
```

Numerical representation

Hybrid filtering uses reflected boundaries and signed float64 values. Intermediate bands are neither clipped nor contrast-normalized. Display exports clamp to [0, 1] and quantize to 8 bits; the settings table counts clipped scalar channel values. The signed high-pass display is shifted by 0.5 and includes the synthesis gain, without additional display amplification.

These excerpts show the filtering and reconstruction steps. The source ZIP contains the complete alignment, masking, validation, and figure-generation code.

SUPPORTING INFORMATION

R Reproduction and observations

Observations

Spatial scale determines which structures are preserved. Smoothing suppresses fine variation before differentiation; hybrid images separate broad shape from fine detail; multiresolution blending uses broader transitions for brightness and color than for texture. Numerical agreement checks confirm the implementations, but alignment and source geometry still limit the visual results.

Reproduce the experiments

The source ZIP includes the original inputs, parameters, implementations, tests, measurements, and report builder. The following commands regenerate the experiments and webpage. README.md describes PDF export and image replacement.

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
python -m unittest discover -s tests -v
python run_filters.py
python run_frequencies.py
python build_report.py
python -m http.server 8000 --directory site
```

Recorded numerical results are in measurements.json. Source URLs, rights statements, original dimensions, and file hashes are in the image manifest.

References

1. A. Oliva, A. Torralba, and P. G. Schyns. Hybrid images. ACM Transactions on Graphics 25(3), 527–532, 2006.
2. P. J. Burt and E. H. Adelson. A multiresolution spline with application to image mosaics. ACM Transactions on Graphics 2(4), 217–236, 1983.
3. R. Szeliski. Computer Vision: Algorithms and Applications, second edition. Figure 3.42.
4. SYDE 671 Assignment 2, “Fun with Filters and Frequencies.” The required experiments follow the supplied handout, adapted from Berkeley CS180.

S Image credits

The self portrait and street photograph are the supplied Assignment 1 images. The Taj Mahal, Derek/Nutmeg pair, and original apple and orange PNGs are supplied course samples. The custom examples use the credited source photographs.

Image	Credit	Rights / provenance	Link
Derek portrait	Derek Hoiem / course materials	Course sample; see note below.	Source
Nutmeg the cat	Derek Hoiem / course materials	Course sample; see note below.	Source
Taj Mahal	Course materials; individual photographer not specified	Course sample; see note below.	Source
Classic cameraman	Course materials; individual photographer not specified	Course sample; see note below.	Source
Apple (supplied Burt-Adelson original)	Peter J. Burt and Edward H. Adelson / course materials	Course sample; see note below.	Supplied Part 2 image package
Orange (supplied Burt-Adelson original)	Peter J. Burt and Edward H. Adelson / course materials	Course sample; see note below.	Supplied Part 2 image package
Moksh portrait	User-supplied photograph	User-provided photograph.	Supplied photograph
Street photograph	User-supplied photograph	User-provided photograph.	Supplied photograph
Astronaut Eileen Collins portrait	NASA	Public domain (NASA photograph)	Source License

Course samples are supplied for this assignment; no general open license is asserted. Personal photographs are user-provided.

S Image credits, continued

The following source records complete the input-image manifest. Adapted figures retain the applicable attribution and license information.

Image	Credit	Rights / provenance	Link
The Blue Marble / Earth seen from Apollo 17	NASA / Apollo 17 crew	Public domain (U.S. government work)	Source License
Full Moon, 22 October 2010	Gregory H. Revera	CC BY-SA 3.0	Source License
Barack Obama official portrait, December 2012	Pete Souza / White House	Public domain (U.S. government work)	Source
Young male tabby cat, Portugal, November 2010	Joaquim Alves Gaspar (Alvesgaspar)	CC BY-SA 3.0	Source License
Front-facing orange-and-white cat, Bengaluru	Himanshu Choudhary (@aalochak)	Unsplash License	Source License

Adapted figures containing the Moon photograph by Gregory H. Revera or the tabby photograph by Alvesgaspar remain under CC BY-SA 3.0. Complete attribution and modification records accompany the code and image manifest.